

Contact Us

Criterion Software Ltd.

For general information about RenderWare Graphics e-mail info@csl.com.

Developer Relations

For information regarding Support please email devrels@csl.com.

Sales

For sales information contact: rw-sales@csl.com

Acknowledgements

Contributors	RenderWare Graphics Development and Documentation Teams
--------------	---

The information in this document is subject to change without notice and does not represent a commitment on the part of Criterion Software Ltd. The software described in this document is furnished under a license agreement or a non-disclosure agreement. The software may be used or copied only in accordance with the terms of the agreement. It is against the law to copy the software on any medium except as specifically allowed in the license or non-disclosure agreement. No part of this manual may be reproduced or transmitted in any form or by any means for any purpose without the express written permission of Criterion Software Ltd.

Copyright © 1993 - 2002 Criterion Software Ltd. All rights reserved.

Canon and RenderWare are registered trademarks of Canon Inc. Nintendo is a registered trademark and NINTENDO GAMECUBE a trademark of Nintendo Co., Ltd. Microsoft is a registered trademark and Xbox is a trademark of Microsoft Corporation. PlayStation is a registered trademark of Sony Computer Entertainment Inc. All other trademark mentioned herein are the property of their respective companies.

Table of Contents

Part C - Animation libraries	7
Chapter 13 - Skinning.....	9
13.1 Introduction	10
13.2 Creating Skinning Data	11
13.2.1 Attaching the RpSkin plugin	11
13.2.2 Creating the RpSkin data	11
13.2.3 Node IDs	16
13.2.4 Destroying the RpSkin data.....	16
13.2.5 Querying the RpSkin data	16
13.3 Using Skinning.....	18
13.3.1 The RpSkin Object	18
13.4 Examples	20
Chapter 14 - Fundamental Types for Animation	21
14.1 Introduction	22
14.2 Quaternions	23
14.2.1 Usage.....	23
14.3 Spherical Linear Interpolation	27
14.3.1 Applications	27
14.3.2 Usage.....	28
14.4 Summary.....	30
14.4.1 Quaternions	30
14.4.2 Spherical Linear Interpolation.....	30
Chapter 15 - Morphing.....	31
15.1 Introduction	32
15.1.1 What Morphing Is	32
15.1.2 What Morphing is Not.....	32
15.1.3 Basic Concepts	33
15.1.4 Strengths and Weaknesses	33
15.1.5 Other Documents.....	34
15.2 Morphing Structures	35
15.2.1 Geometry	35
15.2.2 Atomic	35
15.2.3 Morph Targets	36
15.2.4 Interpolators	36
15.3 How to Morph a Geometry.....	38
15.3.1 Before Adding a Morph Animation	38
15.3.2 How To Set Up Morph Data	38
15.3.3 Animating the Morph.....	40
15.3.4 Effects and Variations.....	40
15.3.5 Destruction	40

15.4 The Morph Example.....	41
15.5 Summary.....	43

Chapter 16 - Delta Morphing 45

16.1 Introduction	46
16.1.1 Morphing & Delta Morphing.....	46
16.1.2 DMorphing.....	46
16.1.3 Animation.....	46
16.1.4 Examples.....	47
16.2 Basic DMorph Usage.....	48
16.2.1 Loading a pre-built example.....	48
16.2.2 Animating.....	49
16.3 RpGeometry and RpDMorphTargets.....	50
16.3.1 RpGeometry	50
16.3.2 Adding RpDMorphTargets	50
16.3.3 Saving DMorph RpGeometry	51
16.3.4 Direct control of DMorph Values	51
16.3.5 Tranforming RpGeometry with RpDMorphTargets Attached	52
16.3.6 Destroying RpDMorphTargets.....	53
16.4 Animation	54
16.4.1 Creating Frames	54
16.4.2 Saving Animations	55
16.4.3 Editing and Querying Frame Sequences.....	55
16.4.4 Loop CallBacks	55
16.4.5 Running an Animation	55
16.4.6 Destroying Frames.....	56
16.5 Summary.....	57
16.5.1 Delta Morphing.....	57
16.5.2 Basic Usage	57
16.5.3 RpGeometry and RpDMorphTargets	57
16.5.4 RpDMorphAnimation	58

Part D - Special Effects Libraries 59

Chapter 17 - The Material Effects Plugin 61

17.1 Introduction	62
17.1.1 How RpMatFX Works	62
17.1.2 RpMatFX Features.....	62
17.2 Using Material Effects	63
17.2.1 Selecting The Effect	63
17.2.2 Initializing Effect Data	63
17.2.3 Enabling the Effects Renderer	70
17.3 Examples	72
17.3.1 A Bump Mapping Example	72
17.4 Summary.....	74
17.4.1 Supported Effects	74
17.4.2 Extended Objects	74

Chapter 18 - PTank.....	75
18.1 Introduction	76
18.1.1 What is a Particle?	76
18.1.2 What Are Particles Used For?.....	76
18.1.3 What Is the Particle Tank?	78
18.1.4 What Particles Are Not.....	78
18.1.5 Other Documents.....	79
18.2 The Main Concepts	80
18.2.1 The Particle.....	80
18.2.2 The Particle Tank	85
18.2.3 RpPTankLockStruct	86
18.2.4 RpPTankFormatDescriptor.....	86
18.2.5 Locking and Unlocking.....	87
18.3 How to Use Particles Step by Step.....	91
18.3.1 Initialization	91
18.3.2 Defining Particles.....	91
18.3.3 Animation	93
18.4 Examples	95
18.5 Troubleshooting	96
18.6 Summary.....	97
 Chapter 19 - B-splines and Bézier Patches	 99
19.1 Introduction	100
19.2 B-splines.....	101
19.2.1 Introduction	101
19.2.2 What Are B-splines?	101
19.2.3 Some Features of B-splines.....	102
19.2.4 Why Use B-splines?.....	105
19.2.5 How RenderWare Graphics Processes Two-dimensional B-spline Curves. 106	
19.2.6 Spline Summary	109
19.3 3D Bézier Patches	110
19.3.1 Introduction	110
19.3.2 What Are Patches?	110
19.3.3 Why Use Patches?.....	111
19.3.4 How RenderWare Graphics Handles Patches	112
19.3.5 How To Use Patches	117
19.3.6 Example Code	127
19.3.7 Summary	128
19.4 Bézier Toolkit	129
19.4.1 Introduction	129
19.4.2 Data Types	130
19.4.3 Quad Patch from Tri Patch	131
19.4.4 Surface Points to Control Points and Back	132
19.4.5 Forward Differencing	134
19.4.6 Patch Tangents and Normals	137
19.4.7 Toolkit Summary	139

19.5 Summary	140
Part E - World Management Libraries	141
Chapter 20 - Collision Detection	143
20.1 Introduction	144
20.1.1 Plugins & Toolkits	144
20.2 Detecting Collisions	145
20.2.1 The RpCollision Plugin	145
20.2.2 The RtIntersection Toolkit	145
20.3 Picking	147
20.3.1 The RtPick Toolkit	147
20.4 Static Geometry Intersections	149
20.4.1 Collisions with World Triangles	149
20.4.2 Collisions with World Sectors	151
20.4.3 Collisions with World Atomics	151
20.5 Atomic & Geometry Intersections	152
20.5.1 Collision Data	152
20.5.2 Performing Collision Tests	152
20.5.3 Example	154
20.6 Summary	155
20.6.1 APIs	155
20.6.2 Hints & Tips	155
Chapter 21 - Potentially Visible Sets	157
21.1 Introduction	158
21.1.1 What are Potentially Visible Sets?	158
21.1.2 The APIs	158
21.1.3 Applications for PVS functionality	158
21.1.4 Reasons for NOT using PVS data	159
21.2 Building PVS Data	160
21.2.1 Using the PVS Converter	160
21.2.2 Using the Static World Viewer	160
21.2.3 Using the Sector Editor	160
21.2.4 Using RpPVS	161
21.2.5 Using RtSplinePVS	163
21.2.6 Generation Progress CallBacks	164
21.3 Using PVS Data	166
21.3.1 Unhooking the PVS Subsystem	166
21.3.2 PVS Runtime Utility Functions	167
21.3.3 Writing Your Own PVS Render CallBack Function	168
21.4 Summary	169
21.4.1 Potentially Visible Sets	169
21.4.2 Generating PVS Data	169
21.4.3 Rendering	170
Index	171

Part C

Animation libraries

Chapter 13

Skinning

13.1 Introduction

Skinning provides a means to animate a model while reducing the "folding" and "creasing" of polygons as the model animates.

The process begins by defining a bone hierarchy in a model. The bone hierarchy is linked to the model's mesh, so that animating the bone hierarchy animates the mesh's vertices.

Up to four bones affect each vertex in the skin mesh. Each bone has a weighting value to determine how much influence it has in proportion to the others.

Skinning is supported by RpSkin and is covered in this chapter. RpHAnim, which implements a full-featured hierarchical animation system, also relies on RpSkin to provide skinning support for its own animation system. Details of using RpHAnim are covered in The Hierarchical Animation Plugin Chapter .

The RpSkin plugin uses the RpHAnim hierarchy to define how the bones of the model move. These bones are linked to vertices within an atomic with a matching structure so that animating the bones also animates the atomic.

It is important to understand that the hierarchy is a distinct entity, separate from the atomic. If their respective structures match, multiple hierarchies can be attached to a single skinned model. Similarly, multiple models can also use the same hierarchy.

13.2 Creating Skinning Data

Skinning data involves the generation of two sets of data:

1. The `RpSkin` data resides in a `.DFF` file, saved using the streaming of the `RpGeometry` that the `RpSkin` is attached to. This defines the relationship between the bone hierarchy and the skin.
2. The `RpAnimHierarchy` animation data is stored in a separate `.ANM` file. This defines the animation key-frames that each bone takes during its animation.

Any number of `RpAnimAnimation` animations can be applied to a single `RpAnimHierarchy` controlling the skin's bones, providing that the bone hierarchy itself remains the same. This is because `RpAnimAnimation` animations are not explicitly linked to model data. So, as long as the model to which the animation is applied has a matching structure, the data will be valid.

It is possible to apply different animations to a hierarchy. The model should be exported once and the different animations should then be exported individually. A single `.DFF` and a number of `.ANM` files will be produced.

13.2.1 Attaching the `RpSkin` plugin

Before skinning is supported, the `RpSkin` plugin must be attached by calling the function `RpSkinPluginAttach()`. This registers the `RpSkin` extension to an `RpGeometry` (automatic streaming, constructor extension, and cloning functions with RenderWare Graphics).

13.2.2 Creating the `RpSkin` data

In most situations, the skinning data will be created at the model export stage. The key function for this purpose is `RpSkinCreate()`.

This function takes the following parameters:

The number of vertices in the skin.

The number of bones in the skin.

An array of vertex weights, one per vertex.

An array of vertex indices, one per vertex.

An array of inverse matrices.

These parameters are explained below:

Number of vertices

To reserve memory for the skin, the number of vertices in the skin (normally equal to the number of vertices in the geometry mesh that is going to be skinned) is passed as a parameter.

Number of Bones

This value is extracted from the modeler when using the RenderWare Graphics exporter. This value is used as the array size for the array of inverse bone matrices. (See the note below for details of platform specific limitations.)

PlayStation 2

The maximum number of bones supported by RpSkin alone on the PS2 platform is 64, due to a limited amount of VU1 memory being available. The restriction is made for performance reasons.

The RpHAnim plugin provides a means of bypassing this limitation through its support of sub-hierarchies.

Xbox

The maximum number of bones supported by RpSkin alone on the Xbox platform is slightly more involved. The absolute upper limit on the number of bones per skin on the hardware is 64 because there are only 192 constant slots and 3 slots are needed per bone.

However, we also need 7 constant slots for transforms and material color. So the maximum number of bones is reduced to $(192 - 7) / 3 = 61$ for anything practical.

Lights also require constant slots:

- 2 each per direction light,
- 2 each per point light,
- and the ceiling (number of point lights / 4).

Material effects also takes up constant slots:

- Environment mapping requires 2,
- Bump mapping requires 7, and
- Bump mapping and environment mapping requires 7.

So for example rendering a character with 8 point lights, 8 directional lights, with bump and environment mapping allows enough space for:

$$(192 - 7 - 2 * 8 - 2 * 8 - \text{ceiling}(8 / 4) - 7) / 3 = 48 \text{ bones.}$$

Only the bones that are used by a particular skin are remapped into the bones array. Hence it is possible to have more than 64 bones per hierarchy as long as each skin in that hierarchy satisfies:

7 (overhead for transforms and material properties)

+ 3 * number of bones

+ 2 * number of directional lights

+ 2 * number of point lights

+ ceiling (number of pointer lights / 4)

+ matfx slots

<= 192 (number of constant registers in the Xbox).

Vertex Weights

An array of `RwMatrixWeights` is passed to `RpSkinCreate()` as a parameter. The array length is determined as the number of vertices for which the skinning data is being created.

Each `RwMatrixWeights` structure is made up of four `RwReals`. Each represents the weight of the corresponding bone in the array of vertex indices.

A weight may contain a value in the range 0.0 to 1.0. The sum of the four weights affecting the vertex should be 1.0 under normal conditions and, although values not totaling 1.0 give an officially undefined result, experience has shown that others can have interesting results.

Processing of weights for a bone will stop when the first weighting of 0.0 is found, and we assume that there are no more bones affecting the vertex. All the unused weights should be set to 0.0. The vertex weights must therefore be arranged such that all valid weights are first in the `RwMatrixWeights` structure and all zero weights are last. This can be done by sorting the weights into descending order. The vertex indices should reflect this sorted order.

The weights in the vertex weights array must be arranged in the same order as the vertices in the atomic.

Bone Vertex Indices

Each vertex can be affected by up to four bones, and the bone ID of each of the four bones are stored in here. The `RwUInt32` contains four packed `RwUInt8` values, each one of which contains an integer 0 to 255, so the maximum number of bones supported in a skeleton is 256.

The following macro will pack the indices into the bone vertex:

```
#define PACK(b1, b2, b3, b4) ( (b4 << 24) + (b3 << 16) + \
                               (b2 << 8) + b1 )
```

Inverse Bone Matrices

The transformation from skin space to bone space, required at run-time for modifying vertex positions, is achieved by transforming the vertex position by the Inverse Bone Matrix (IBM). Since the root of the hierarchy is attached to the mesh in an `RwFrame` hierarchy, the IBM is the inverse of the overall transformation matrix—the Local Transformation Matrix (LTM)—from the mesh to the bone.

The `RpSkinCreate()` function requires an array of IBMs, stored in bone order. The length of the array is determined from the number of bones.

Within the hierarchy, `RwFrames` are stored for each bone node, indicating the transformation of that bone with respect to its parents.

The run-time procedure then performs the following operations:

1. Transforms vertex positions from their object space (skin space) into bone space using the IBMs.
2. Performs the key-frame based animation for that frame, which transform the vertex positions back into skin space in it's animated pose.
3. The model is then rendered.

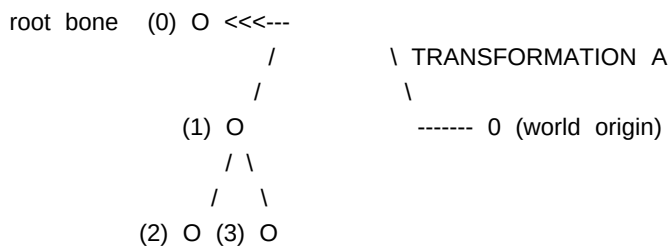
The inverse bone matrices are `RwMatrix` 's. There should be one per bone, i.e. the same number as you are passing to the `numBones` parameter of `RpSkinCreate()` . The inverse bone matrices describe the transformation matrix from each bone in the hierarchy to the root bone in the hierarchy.

To find the inverse bone matrix, one requires the transformation matrix from the bone to the root bone.

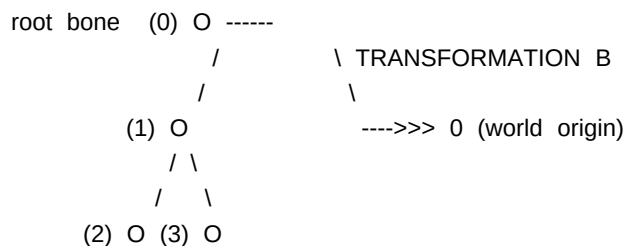
For instance for bone 3 this is obtained by:

(The results from each of the following 4 steps is shown on the diagrams.)

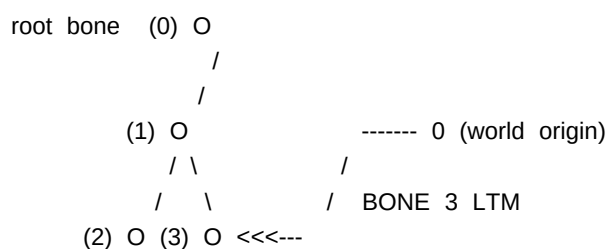
1. finding the transformation matrix from the world origin to the root bone,

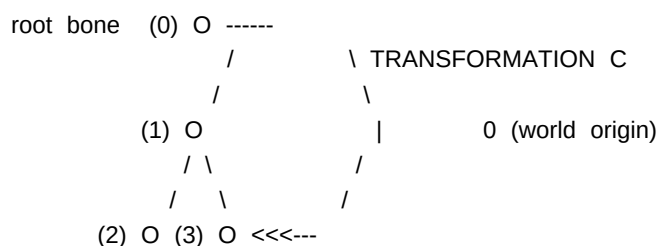


2. inverting this to find the transformation matrix from the root bone to the world origin,

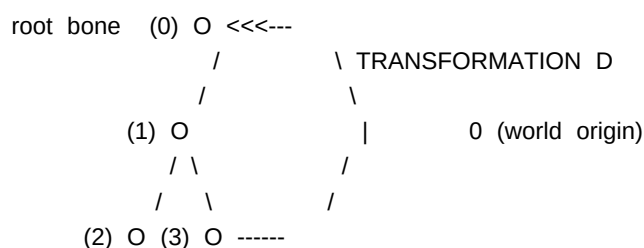


3. multiplying this by the LTM of bone 3 to find the transformation from the root bone to bone 3,





4. inverting this to find the transformation from bone 3 to the root bone.



The following code calculates the inverse bone matrix if you have the world transformation matrix for each bone. Calculate for each bone in turn.

```

{
    /*
     * INPUTS: node is a structure with the current
     *         bone world transformation stored in
     *         a frame root is a structure with the
     *         root bone world transformation stored
     *         in a frame.
     * OUTPUT: ibm is a pointer to the inverse bone
     *         matrix of the current node.
     */

    RwMatrix *invHierarchyRootMatrix;
    RwMatrix *rootRelativeMatrix;
    RwMatrix *LTM;
    RwMatrix *rootLTM;

    /* Calculate & store the inverse bone matrix *
     * within the hierarchy */

    invHierarchyRootMatrix = RwMatrixCreate();
    rootRelativeMatrix = RwMatrixCreate();
    LTM = RwFrameGetLTM(node->frame);
    rootLTM = RwFrameGetLTM(root->frame);

    RwMatrixInvert(invHierarchyRootMatrix, rootLTM);
    RwMatrixMultiply( rootRelativeMatrix,
                      LTM,
                      invHierarchyRootMatrix );
}
  
```

```
RwMatrixInvert(ibm, rootRelativeMatrix);  
  
RwMatrixDestroy(invHierarchyRootMatrix);  
RwMatrixDestroy(rootRelativeMatrix);  
}
```

13.2.3 Node IDs

The array of nodes in RpHAnim equates to the array of bones in RpSkin . The node IDs are usually generated automatically by the RenderWare Graphics exporter for the modeling package. An artist can, however, override the defaults and use their own IDs for a particular node (which relate directly to the bones). This facility makes it easier for programmers to locate any bones that require special treatment, as would be the case for many procedural animation techniques.

The node IDs must be unique within the hierarchy. Normally, a modeling package will supply unique bone IDs, but in the event that they are not tagged, unique IDs should be created dynamically within the exporter by default.

Assigning Node IDs within a Modeling Package

Support for RpSkin and RpHAnim is available for both the 3ds max and Maya modeling packages. As each modeling package has a different user interface, package-specific details on tagging objects can be found within the appropriate Artist Guide for your preferred package.

Extracting the Node ID at Run-time

The RpHAnimIDGetIndex() function returns the index from an RpHAnimHierarchy for a particular Node ID. This index maps directly into the skin to bone matrix array. The skin to bone matrix array is retrieved with RpSkinGetSkinToBoneMatrices() . (Elements of the matrix array are of type RwMatrix .)

13.2.4 Destroying the RpSkin data

The RpSkin data is destroyed with RpSkinDestroy() , all internal and platform specific data is cleaned up automatically.

13.2.5 Querying the RpSkin data

The following functions give read-only access to the RpSkin data.

RpSkinGetNumBones() return the number of bones in the RpSkin .

RpSkinGetVertexBoneWeights() return the array of vertex bone weights (RwMatrixWeights *) in the RpSkin .

`RpSkinGetVertexBoneIndices()` return the array of packed vertex bone indices (`RwUInt32 *`) in the `RpSkin` .

`RpSkinGetSkinToBoneMatrices()` return the array of skin to bone matrices (`RwMatrix *`) in the `RpSkin` .

13.3 Using Skinning

When using skinning, there are three main objects to deal with:

The `RpSkin` object

The `RpHAnimHierarchy` object

The `RpHAnimAnimation` object

The use of `RpHAnimHierarchy` objects and `RpHAnimAnimation` objects is described in The Hierarchical Animation Plugin Chapter .

13.3.1 The `RpSkin` Object

The `RpSkin` object contains the skinning data. The vertex weights, vertex indices and IBMs are all used by the skin rendering pipeline.

The `RpSkin` object should be attached to an `RpGeometry` with `RpSkinGeometrySetSkin()` . The `RpGeometry` vertices match up to the skinning vertex information in the `RpSkin` .

The `RpSkin` object attached to an `RpGeometry` can be retrieved with `RpSkinGeometryGetSkin()` .

When an `RpGeometry` is streamed in as part of the .DFF file, any attached `RpSkin` will also be streamed in. Likewise, any attached `RpSkin` will automatically be streamed out with an `RpGeometry` .

It is possible that there will be multiple `RpAtomic` s referencing the same `RpGeometry` instance. So the skin's animation data is attached to the `RpAtomic` instead of the `RpGeometry` . The hierarchy is attached to the atomic with `RpSkinAtomicSetHAnimHierarchy()` . An `RpAtomic` 's presently attached hierarchy can be retrieved with `RpSkinAtomicGetHAnimHierarchy()` .

Attaching the `RpSkin` to an `RpGeometry` , and then attaching an `RpHAnimHierarchy` to the `RpAtomic` referencing the `RpGeometry` , has fully setup the data required for skinning the mesh. However the rendering pipeline attached to the `RpAtomic` needs to be overloaded with a custom skinning pipeline. The default rendering pipeline knows nothing about the skinning data extension.

The `RpSkinType` enumeration lists the different rendering pipeline types available within the `RpSkin` plugin. At present these are:

`rpSKINTYPEGENERIC`– pipeline renders generic skinned geometry.

`rpSKINTYPEMATFX`– pipeline renders material effected skinned geometry.

A skinned rendering pipeline is attached to the `RpAtomic` with `RpSkinAtomicSetType()`. The type of the present skinning pipeline can be queried from an `RpAtomic` with `RpSkinAtomicGetType()`.

RpSkin Libraries: `rpskin.lib` and `rpskinmatfx.lib`

There are two versions of the `RpSkin` libraries in the RenderWare Graphics SDK. They are both fully featured versions on the `RpSkin` plugin, and they contain identical APIs. However, because the rendering pipelines are large we've taken the step to compile different versions of the plugin so that the user can select precisely the pipelines they will be using.

The `rpskin.lib` library only contains the `rpSKINTYPEGENERIC` pipeline.

Whereas the `rpskinmatfx.lib` library contains both the `rpSKINTYPEGENERIC` and `rpSKINTYPEMATFX` pipelines.

Only one of the skinning libraries should be used in an application at once.

RpSkin & RpPatch

The `RpPatch` plugin also supports skinned patch meshes. The `RpPatchMeshes` are used with the skinning plugin in a very similar ways to `RpGeometry`s.

`RpPatchMeshSetSkin()` should be used instead of `RpSkinGeometrySetSkin()`, and

`RpPatchMeshGetSkin()` should be used instead of `RpSkinGeometryGetSkin()`.

Once the `RpPatchMesh` has been attached to the `RpAtomic` (in place of an `RpGeometry`), the correct skinning patch rendering pipeline must be attached to the `RpAtomic`. There are two skinning patch rendering pipelines in the `RpPatch` plugin:

`rpPATCHTYPESKIN`– pipeline renders skinned patches.

`rpPATCHTYPESKINMATFX`– pipeline renders skinned material effected patches.

The `RpPatch` pipelines are attached with `RpPatchAtomicSetType()` instead of `RpSkinAtomicSetType()`.

For more details on using the skinning patch pipeline please read [The B-splines and Bézier Patches Chapter](#).

13.4 Examples

The core skinning features exposed by `RpSkin` are used by the `RpHAnim` examples listed below.

`hanim1`

`hanim2`

`hanim3`

`hanimkey`

Chapter 14

Fundamental Types for Animation

14.1 Introduction

The RenderWare Graphics SDK contains support for quaternions (`RtQuat`), which provide functionality for orienting key-frames, and spherical linear interpolators (`RtSlerp`), which provide functionality for interpolating smoothly between these key-frames.

This chapter does not attempt to give an in-depth description of quaternions and Slerps, concentrating instead on detailing the support provided for them.

For further information on these subjects, see the resources listed in the Recommended Reading appendix.

14.2 Quaternions

The `RtQuat` toolkit exposes support for quaternions.

A quaternion consists of four elements; a real value and three imaginary values. Together, these can represent both scaling and rotation transforms, with the advantage that operations always produce orthogonal results—i.e., there is no shearing created by rounding errors, such as can occur when using matrices alone.

Quaternions are compact, compared to the nine values needed for a matrix, so they are useful on platforms where memory is at a premium or where the architecture favors smaller data structures.

The `rtquat.h` header file needs to be included and its library linked against.

14.2.1 Usage

Creation

Quaternions are small enough to be declared as automatic variables. They are initialized, rather than being declared as pointers and being allocated memory. For this reason, they don't need constructor or destructor functions and the `RtQuat` structure is transparent and can be addressed directly.

An `RtQuat` structure contains two elements:

`real` – an `RwReal` value representing the real value

`imag` – an `RwV3dvector` representing the imaginary values

These can be modified directly if required, although an `RtQuatInit()` function is provided by the API for convenience.

The following code fragment shows a unit quaternion being created and initialized:

```
RtQuat myQuat;  
  
...  
  
RtQuatInit( &myQuat, 0.5f, 0.5f, 0.5f, 0.5f );
```

Rotations

Quaternions are usually used to perform rotations. To this end, the `RtQuat` API includes the `RtQuatRotate()` to initialize a quaternion using a vector which represents the axis of rotation, and an angle representing the rotation itself.

Conversely, the `RtQuatQueryRotate()` function will return the axis and angle from a given quaternion.

Scaling

The length, or modulus , of a quaternion represents the scale. Although only rotation is usually required of a quaternion, the `RtQuatScale()` function is provided to allow the application to re-scale a quaternion, into a new quaternion.

Transforming Vectors

A quaternion rotation is often applied to a vector array. For this purpose, the `RtQuat` API provides the `RtQuatTransform()` function, which applies the transformation represented by a quaternion to an array of vectors.

API

The `RtQuat` toolkit provides a full-featured API and includes a number of quaternion operations. The operations and matching functions names are listed in the table below:

FUNCTION	PURPOSE
<code>RtQuatAssign()</code>	Copies one quaternion to another.
<code>RtQuatConjugate()</code>	Negates the imaginary parts of a quaternion.
<code>RtQuatConvertToMatrix()</code>	Takes a quaternion and converts it to the equivalent matrix.
<code>RtQuatConvertFromMatrix()</code>	Takes a matrix and returns the equivalent quaternion.
<code>RtQuatAdd()</code>	Adds two quaternions together, producing a third. ($A=B+Q$)
<code>RtQuatSub()</code>	Calculates the difference between two quaternions. ($A=B-C$)
<code>RtQuatIncrement()</code>	Increments a quaternion by another. (Equivalent to $A+=B$)
<code>RtQuatDecrement()</code>	Decrements a quaternion by another. (Equivalent to $A-=B$)
<code>RtQuatIncrementRealPart()</code>	Increments the real part of a quaternion by a specified real value.
<code>RtQuatDecrementRealPart()</code>	Decrements the real part of a quaternion by a specified real value.
<code>RtQuatMultiply()</code>	Calculates the (non-commutative) product of two quaternions.
<code>RtQuatNegate()</code>	Negates a quaternion to the additive inverse.
<code>RtQuatModulus()</code>	Returns the modulus—the scaling component—of a quaternion.
<code>RtQuatModulusSquared()</code>	Returns the square of the modulus of a quaternion.
<code>RtQuatExp()</code>	Calculates the exponential of a quaternion.
<code>RtQuatPow()</code>	Calculates the power of a quaternion.
<code>RtQuatLog()</code>	Calculates the logarithm of a quaternion.

RtQuatQueryRotate()	Determines the rotation represented by a quaternion. The rotation is returned as a unit vector along the axis of rotation, and an angle of rotation. The rotation component has two possible descriptions since a rotation about an axis of theta degrees is equivalent to a rotation about an axis pointing in the opposite direction by an angle of $360^\circ - \text{theta}$ in the reverse direction. The rotation with the smaller angle is returned.
RtQuatRotate()	Builds a rotation quaternion from the given axis and angle of rotation.
RtQuatScale()	Scales a quaternion by the specified factor.
RtQuatSquareRoot()	Calculates the square root of the specified quaternion.
RtQuatTransformVectors()	Uses the given quaternion describing a transformation and applies it to the specified array of vectors. The results are then placed in another array (which may be the same array as the source).
RtQuatUnitConvertToMatrix()	Converts from a unit quaternion to a matrix.
RtQuatUnitExp()	Calculates the exponential of a unit quaternion.
RtQuatUnitLog()	Calculates the logarithm of a unit quaternion.
RtQuatUnitPow()	Calculates the power of a unit quaternion.
RtQuatReciprocal()	Reciprocates a quaternion to its multiplicative inverse.

14.3 Spherical Linear Interpolation

The `RtSlerp` toolkit exposes support for spherical linear interpolations—so-called "Slerps". The `rtslerp.h` header file needs to be included and its library linked against.

This toolkit works closely with quaternions so it also requires `RtQuat` to be linked in to your application.

14.3.1 Applications

Slerps are used to spherically interpolate between two quaternions—each typically representing a key-frame orientation. This is a common requirement in animations where linear interpolation is not always appropriate—animating skinned models, for example.

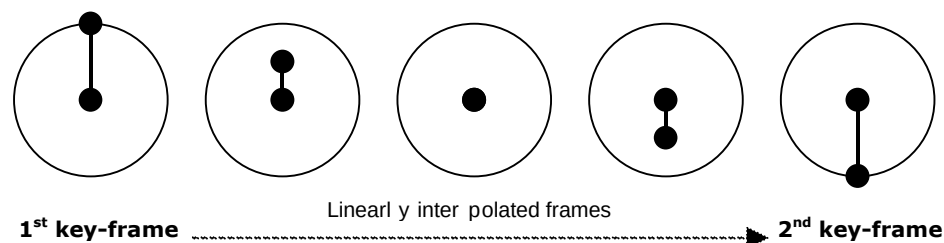
The primary use for both quaternions and Slerps is interpolating rotations in animation. Quaternions are often used together with spherical linear interpolators to interpolate rotations along an arc; the result is generally more "natural" than a linear interpolation.

RenderWare Graphics supports Slerps through the `RtSlerp` toolkit, which is covered in Section 1.3 of this chapter.

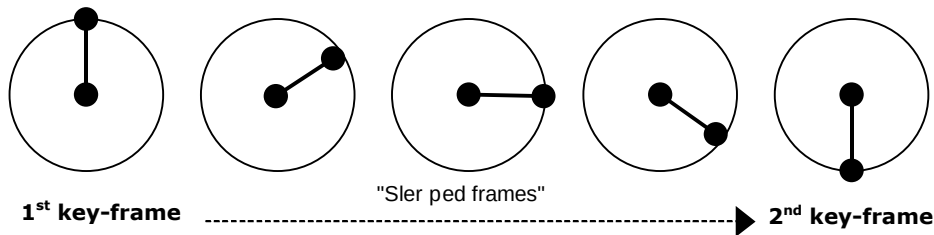
Why not use Morph Targets?

Morph target animation uses linear interpolation exclusively. This means that the interpolated frames created between one key-frame and the next are always positioned in a straight line. This is great for animating, say, a spaceship. But many situations call for interpolating in an arc, rather than a straight line.

A simple example is an analog clock face. The following sequence shows how the hands would move if linear interpolation is used between the two key-frames:



Because Slerps interpolate the animation along an arc rather than a straight line, the clock face above would be animated as shown below. (The frames are said to have been "Slerped", which is an abbreviation for "Spherical-linearly interpolated").



The most common use of Slerps is for skinned animations based around virtual "bones". In such animations, the bones must be animated such that the paths describe curves in space, rather than lines—i.e. the bones maintain their length throughout the interpolation.

14.3.2 Usage

Creation

Slerps are represented by the `RtSlerp` object. This object is transparent and defined as follows:

`angle` – The angle, in degrees, between source and destination. (`RwReal`).

`axis` – The axis of rotation for the Slerp. (`RwV3d`).

`endMat` – The end matrix. (`RwMatrix *`).

`matRefMask` – Flags specifying which matrices are not managed by the Slerp object. (`RwInt32`).

`startMat` – The start matrix. (`RwMatrix *`).

`useLerp` – If `TRUE` then linear interpolations will be used. (`RwBool`).

The constants listed below are used with the `matRefMask` element:

`rtSLERPREFNONE`– Both start and end matrices are copied into the structure rather than accessed by reference.

`rtSLERPREFSTARTMAT`– The start matrix is referenced and should be destroyed by the application when no longer required. The end matrix is copied.

`rtSLERPREFENDMAT`– The end matrix is referenced and should be destroyed by the application when no longer required. The start matrix is copied.

`rtSLERPREFALL` – Both start and end matrices are referenced and should be destroyed by the application when no longer required.

Accessing by reference is usually the preferred option, but it should be noted that it is usually necessary for Slerp objects to persist across rendering cycles, so that they can be used to continue the interpolation. Copying of matrices requires more memory, but does have the advantage that persistent Slerps may be easier to maintain using this method.

Creating a valid Slerp usually requires two calls:

The first is to `RtSlerpCreate()` . This function takes one argument defining the flags for the `matRefMask` property of the Slerp. One of the constants listed above should be used for this.

Secondly, the Slerp needs to be initialized with valid data using `RtSlerpInitialize()` . This takes a pointer to a Slerp object and pointers to the two key-frame matrices that are to be used. These matrices will be either copied to, or referenced by, the Slerp object according to the flags set in `RtSlerpCreate()` .

`RtSlerpCreate()` must be called before `RtSlerpInitialize()` in order to ensure the flags are set. If this is not done, the result is undefined.

Caches

Two caching structures are exposed by the `RtSlerp` toolkit: `RtQuatSlerpCache` and `RtQuatSlerpArgandCache` . These are used internally by the API to improve performance—either method can be selected.

Full documentation of these structures can be found in the API Reference.

The required cache must be setup before the Slerp can be used for interpolation. The relevant functions are `RtQuatSetupSlerpCache()` and `RtQuatSetupSlerpArgandCache()` .

Performing Spherical Linear Interpolations

With all the requisite structures initialized, it is now possible to perform the spherical linear interpolation. Two functions are available for this purpose:

`RtQuatSlerp()` , which uses the `RtQuatSlerpCache` and

`RtQuatSlerpArgand()` , which makes use of the `RtQuatSlerpArgandCache()` instead.

14.4 Summary

The RenderWare Graphics SDK contains support for quaternions (RtQuat) and spherical linear interpolators (RtSlerp).

14.4.1 Quaternions

The RtQuat toolkit exposes support for quaternions, which represent both rotation and scaling.

A quaternion consists of four elements: one real part, and three imaginary parts.

Morph target animation uses linear interpolation exclusively.

Quaternions are often used together with spherical linear interpolators—also known as Slerps—to interpolate rotations along an arc. The result generally appears more natural than a linear interpolation. Slerps are implemented by the RtSlerp toolkit.

14.4.2 Spherical Linear Interpolation

The RtSlerp toolkit exposes support for Slerps.

Slerps are represented by the RtSlerp object.

Slerps are used heavily by the RpHAnim and RpSkin hierarchical animation systems to animate and rotate the "bones" that represent the model hierarchy.

Chapter 15

Morphing

15.1 Introduction

The chapter covers the data structures that describe morph sequences. The RpMorph plugin executes this data when it drives the morphing process. The chapter describes the morphing data held in the RpAtomic and RpGeometry, and the structures RpMorphTarget, RpInterpolator, and RpMorphInterpolator. It describes how to implement morphing, from preliminaries, to destruction, with some variations on morphing, and it discusses the "morph" example.

15.1.1 What Morphing Is

RenderWare Graphics supports animation in morphing, delta morphing, rigid body animation and skinning. Morphing is the simplest of these, and is implemented primarily in the RpMorph plugin. Morphing changes an object from one pre-defined shape to another pre-defined shape by simple linear interpolation. This is shown in the example, `RWGraphics\examples\morph`, which shows the world stretching from a globe into one of three egg-shapes and back, in six consecutive morph interpolations.

Morphing works well for animating the objects being destroyed like racing cars crashing, objects being crushed or stretched like cushions or bean bags, objects being deformed like a ball when it bounces, or a simple cartoon character in movement.

The start and end states of morph interpolations are called "morph targets" in RenderWare Graphics, but they are sometimes known as "keyframes" so morphing is also referred to as "keyframe interpolation animation" or "keyframe animation".

15.1.2 What Morphing is Not

RenderWare Graphics supports other forms of animation distinct from morphing.

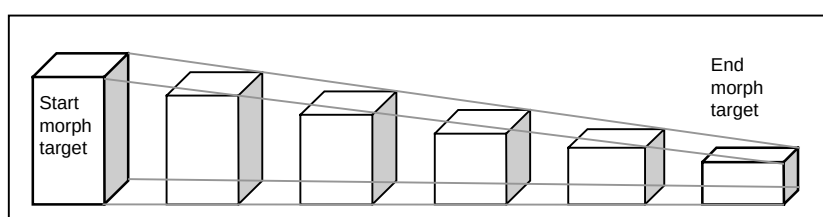
Rigid body animation, in RpHAnim, is different from morphing because it changes the relative positions of objects. Rigid body animation works efficiently when it animates a foot that moves relative to a shin, when the shin moves relative to a thigh which moves relative to a body. Morphing works well for objects that change their shape.

Skinning, in RpSkin, combines elements of rigid body animation and elements of morphing to represent the way that skin stretches over solid forms.

Delta morphing, in RpDMorph, supports changes between multiple shapes at the same time. It is used to change the expression of a sad face into a surprised face that is also sad. It uses one shape for the sad face, and changes it into the a blend of the shape for a surprised face and the shape for a sad face. Morphing is different from delta morphing in that morphing allows changes between no more than two states.

15.1.3 Basic Concepts

Two of the structures that support morphing are defined outside the RpMorph plugin. The RpMorphTarget s are part of RpGeometry , and RpInterpolators are part of the RpAtomic . Both belong to the RpWorld plugin. The array of RpMorphInterpolator s is added to the RpGeometry by the plugin .



Morphing is a linear interpolation between two shapes

In RenderWare Graphics, morphing requires a single starting state and a single end state at any given time. These are called "morph targets", as in the illustration above. Intermediate states are interpolated between them. The interpolation is always linear.

15.1.4 Strengths and Weaknesses

This means that morphing is not well suited for imitating the movement of the hands of a clock because they follow curved paths, and because the image of each hand rotates. Morphing can not represent either effect. The clock face animation could only be approximated by RpMorph in a number of small consecutive morphs between several positions and this would use up memory to store every position. Rigid body animation would be more appropriate.

Morphing can animate between single shapes; a face into a smiling face, or a face into a frown. The RpDMorph plugin was written to interpolate multiple shapes.

Morphing is not limited to simple transformations. Once the vertices of an object and its triangles are defined in a geometry, the form can be morphed into any other form defined by the same vertices and triangles. And a geometry can be morphed successively between a sequence of its morph targets to make up more complicated animation effects.

It should be noted that RenderWare Graphics Bézier patches, are not compatible with morphing.

15.1.5 Other Documents

See the API reference for details of the code for RpMorph and RpWorld .

This chapter assumes you are familiar with the concepts of geometries, atomics, clumps and streaming from their descriptions in this User Guide. They can be found in the Chapters on World and Static Models and in Dynamic Models .

Rigid body animation, delta morphing and skinning can be found in the Chapters on The Hierarchical Animation Plugin , on Skinning , and on Delta Morphing respectively.

15.2 Morphing Structures

Some of the basic data used for morphing is built into the `RpGeometry` and `RpAtomic` objects of the World plugin. The geometry may contain an array of morph targets, the static states, between which the morphs are interpolated. An atomic using such a geometry contains values that specify the current state of morph interpolation for a particular instance of the object.

The `RpMorph` plugin provides a morph animation system by extending the `RpGeometry` and `RpAtomic` objects, to specify a sequence of interpolations between specific morph targets, and the current position within such a sequence.

15.2.1 Geometry

Each `RpGeometry` stores the number of its morph targets. It may hold one or more. There must be at least two morph targets if we are to morph between them. So when a geometry specifies that it has only one morph target this indicates that it cannot support morphing.

The `RpMorph` plugin adds animation extension animation data to geometries, and the extension data holds a pointer to the geometry's morph targets. The morph targets are held in an array, so they are addressed by their index numbers. The morph targets can be used in any combination so that morphs can be defined between any two of them, and any morph target can be used in many morphs.

`RpGeometry` also has an array of `RpMorphInterpolator`s in its animation extension data (the appended data managed by the `RpMorph` plugin). As it is an array, the interpolators are address by their index numbers. Each `RpMorphInterpolator` defines an animation from one morph target to another in a specified number of seconds. The interpolators are linked in one or more sequences that describe an animation through a series of morph targets. Each interpolator points to the next one in the sequence.

15.2.2 Atomic

An `RpAtomic` holds three relevant data items.

- a pointer to its geometry, so it refers to its geometry's morph targets and the morph animation sequence.

- an `RpInterpolator` driven by `RpMorph` that is used to store the current interpolation state. The `RpMorph` plugin updates this as an animation progresses.

the current position within the animation. (The animation is defined by a sequence of `RpMorphInterpolator` s in the geometry). Different atomics sharing a geometry may be at a different points in the animation.

15.2.3 Morph Targets

Morph target structures are part of a geometry. An `RpMorphTarget` is an opaque structure that contains:

- an array of vertices that determine the shape of the morph target. The order of the vertices corresponds to the order of the geometry's own vertex array. The colors, textures, triangle array and other data are held in the geometry and map to the same vertex order. The geometry can hold more shapes for its vertices by adding more morph targets to its array.

- a morph target may also have an array of normal vectors at each vertex, but this is optional.

- a morph target has a bounding sphere that encloses all the vertices in their current positions. This can be used for calculating the effects of light, for collision detection, for culling and for clip tests.

15.2.4 Interpolators

There are two types of interpolator used by the `RpMorph` plugin: `RpMorphInterpolator`, used by the geometry, and `RpInterpolator` used by the atomic.

RpInterpolator

The `RpInterpolator` is part of the atomic. It is opaque and holds the data for the atomic's current morph. Most importantly it stores the 'time' field, that represents the current point in the morph's duration between the two current morph targets. The `RpMorph` plugin updates the time field.

If the developer chooses to use the morph target data without `RpMorph`'s animation facilities then the `RpInterpolator` API Set and Get functions can be used to update an interpolator directly.

RpMorphInterpolator

The `RpMorphInterpolator` does not hold any current state information (as the `RpInterpolator` does) but instead it defines parameters for a single morph interpolation. The parameters will be copied across to the `RpInterpolator` at the appropriate time. So `RpMorphInterpolator` contains

- a pointer to its starting morph target

a pointer to its end morph target

and its duration in seconds (also called its "scale" or "time")

a pointer to the next RpMorphInterpolator in sequence, or to itself if there is none.

An array of RpMorphInterpolator s is added to the geometry when the morph plugin is present. The array stores definitions of single morphs. It allows each interpolator to link on to another. The morph plugin reads this data automatically rendering a sequence of images from one morph to another.

15.3 How to Morph a Geometry

This section deals with preliminaries to implementing a morph interpolation. It describes how morphing data is normally imported from a modeling package, and how the timing is added using API functions. It describes how the interpolation data is made to move, and how movements can be varied and finally, how the extra data is destroyed.

15.3.1 Before Adding a Morph Animation

RpMorph relies on the RpWorld plugin being attached.

Morphing is one of the simplest forms of animation. It is not compatible with the more complicated delta morphing or with skinning, and does not support Bézier patches.

The header file `rpmorph.h`, which needs `rpmorph.rpe`, must be appended to the `#include` list.

`RpMorphPluginAttach()` must be called to attach the RpMorph plugin.

15.3.2 How To Set Up Morph Data

Setting up Morph Data with a Modeling Package

RenderWare Graphics supports the 3ds max or Maya modeling packages, and either of them can be used to create and preview a morph. The Artists Guides for the packages will explain how to export ".dff" files using RenderWare Graphics' exporters. The exporters translate the data from the package for streaming into RenderWare Graphics.

This process replaces some of the developer's tasks. The function `RpClumpStreamRead()` loads the data and unpacks it into

- a clump

- its atomic

- the atomic's geometry

- the geometry's morph targets

- and the geometry's morph interpolators.

This is how it is done in the "Morph" example:

```
if( stream )
{
    RpClump *clump = NULL;

    if( RwStreamFindChunk(stream, rwID_CLUMP, NULL, NULL) )
        clump = RpClumpStreamRead(stream);

    RwStreamClose(stream, NULL);
}
```

This loads all the data from the clump down to the morph targets.

Modifying the Data with the API

Once the exporter has translated the data from the graphics package into internal data structures, the objects are ready to be animated and rendered. The following functions are needed only if the developer wants to alter the behavior of the data from the graphics package.

Sometimes the application will need to replace the exported interpolations, to modify them or to create an animation programmatically. For instance, it might want to animate waves onto a sea using its own algorithm. These functions are exposed for this purpose.

`RpMorphGeometryCreateInterpolators()` reserves space for the number of interpolators required, destroying any interpolators already in place. The number of morph interpolators is passed as a parameter.

`RpMorphGeometrySetInterpolator()` is called once for each morph interpolator and fills or alters all the interpolator's data fields. It sets the interpolator's "next" pointer to the next interpolator in the sequence or to the first, if this is the last morph interpolator in the array.

`RpMorphGeometrySetNextInterpolator()` is provided to override the default behavior of `RpMorphGeometrySetInterpolator()`. It sets or alters the "next" pointer. The "next" pointer determines which morph interpolator in the array of morph interpolators will be executed after this one.

Intercepting the Interpolator Sequence

The callback will be used only in exceptional cases, but if it is needed, this is the point to set the callback function, `RpMorphGeometrySetCallBack()`. It is used to set the callback function that will be executed at the end of each interpolator to trigger some effect or variation.

15.3.3 Animating the Morph

`RpMorphAtomicSetCurrentInterpolator()` sets the animation to the start at the morph interpolator whose index is passed as a parameter. This will usually be zero. Then `RpMorphAtomicSetTime()` is called to set the time, usually zero, within the initial morph interpolator. Now the morph atomic is ready to be rendered.

When updating before rendering a new frame, the function `RpMorphAtomicAddTime()` is called to update the time field in the interpolator and advance to the next interpolator in the sequence when appropriate.

15.3.4 Effects and Variations

The `RpMorphInterpolator` structure is small, and the most efficient way to vary morph animations is to define them as new sequences of morph interpolators. For instance, it is generally more efficient to add an interpolator to reverse a simple morph than it is to swap the start and end morph targets and re-run it.

It is sometimes desirable for a morph to begin slowly, continue at speed and then slow down to its end. This can be done by representing the start and end of the sequence by slower morph interpolators that operate between intermediate morph targets.

In the section above, the geometry's callback function could be set. The callback function is called automatically when each interpolation is completed, and the default callback function simply moves on to the next morph interpolator. The developer can write an alternative callback function to detect intercept a moment in a particular action and trigger an extra effect, and the `RpGeometrySetCallback()` function will select it.

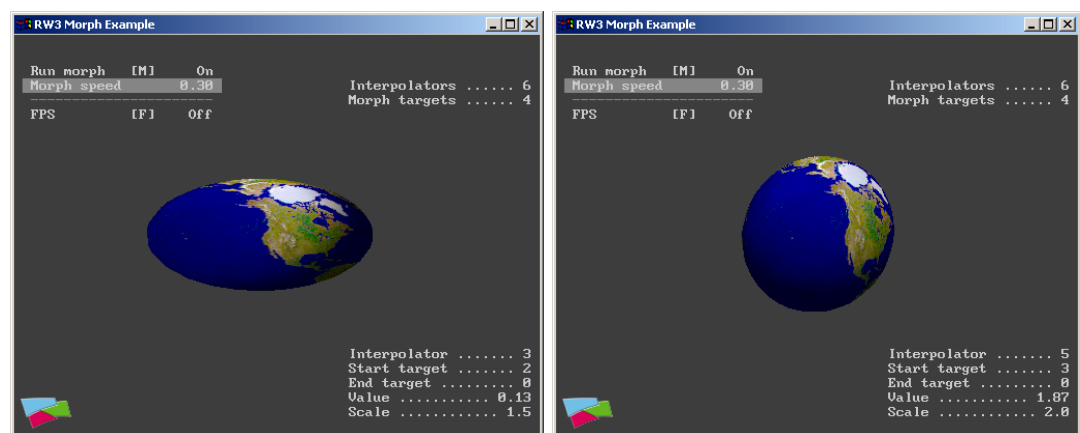
15.3.5 Destruction

If all the structures are set in place correctly, from the clump to the atomic, the geometry and their animation extensions holding morph targets and morph interpolators, then the `RpClumpDestroy` function will destroy all of them in the right order.

15.4 The Morph Example

The RenderWare Graphics example `RW\Graphics\examples\morph` shows a globe constantly changing shape between its default shape and one of three transformations of itself. The transformations are used because they are easy to produce in example code, not because `RpMorph` is limited to morphing between transformations.

The Morph example reads in the file `"world.dff"` in the function `RpClumpStreamRead()` as described above. The data is read in from the file stream. It may also be helpful to note in the code, that the function `RpClumpForAllAtomics()` calls the callback function that updates the geometry with the state for the present frame.



Two screen shots of the Morph example

The up and down cursor keys can be used to de-select the "Morph speed" item. In this state, dragging the mouse or analogue controller rotates the globe demonstrating that the morph is interpolating in 3D. This operation calls the function `ClumpRotate()`.

The "Morph targets" value at the top right of the window shows that there are four morph targets. The "Start target" and "End target" values at the bottom right during the animation will show that the morph targets are numbered 0 to 3. The morph target "0" is the most frequent. It is the default, spherical shape of the earth. All the interpolations morph to or from morph target 0.

The six morph "Interpolators" are numbered 0 to 5 in the field labeled "Interpolator". The globe stretches from its spherical shape (morph target 0) in interpolators 0, 2 and 4, when the "Start Target" is always "0". Conversely when the earth is contracting back to its spherical shape, the "End target" is "0" and the "Interpolator" is 1,3 or 5.

The duration value of each interpolator is shown as the "Scale". Each "Interpolator", 0-5, adopts its own duration each time it is executed.

The "Value" represents the point in time at which each successive interpolation has reached. It increments up to the current value of "Scale". Many of these values, especially this one, will be easier to read if the cursor keys are used to select "Morph speed" to slow down the time scale below "1.0".

15.5 Summary

The RpMorph plugin supports animations built up from linear interpolation between start and end states known as morph targets. Morph targets are variations on the shape of a geometry, and are defined as arrays of vertices that correspond directly with the vertex arrays of their respective geometries.

In an animation sequence, the transition between states is defined by an RpMorphInterpolator over a specified duration, and the vertices are recalculated internally. Each interpolator is stored in an array in a geometry and the interpolation is linear.

The geometry's RpMorphInterpolator array holds the data for one or more interpolation sequences. Each sequence can consist of a single interpolation or a series of consecutive interpolations in which the same morph target that ends one interpolation also begins the next one.

The morphing process implemented by RpMorph is integral to the RpGeometry and to RenderWare Graphics' rendering process.

Morphing is appropriate for linear changes. Rotation and curved paths would have to be approximated by joining sequences of morphs together and other forms of animation in RenderWare Graphics may be more appropriate.

Chapter 16

Delta Morphing

16.1 Introduction

This chapter describes Delta Morphing or "DMorphing" which is supported by the RpDMorph plugin and describes the facilities it provides. (Much of the chapter refers to dynamic models, RpClumps and RpGeometry objects, which are covered in the Dynamic Models chapter.)

In this section, DMorphing is introduced. In Section 16.2 basic DMorph usage is covered and an example loading a pre-built model is presented. In Section 16.3, geometry (RpGeometry) and DMorph targets (RpDMorphTarget) are introduced. Animation (RpDMorphAnimation) is covered in Section 16.4, and the chapter is summarized in Section 16.5.

16.1.1 Morphing & Delta Morphing

Morphing is used to generate the intermediate frames needed to seamlessly morph one geometry to match another. For example, changing a facial expression from a frown to a smile. In use, the developer specifies starting and ending target objects and then functions are used to generate interpolated new geometry data from these two targets over time.

DMorphing differs in that there are a number of targets which may be applied to a base geometry. In RenderWare Graphics, this can be used to generate combinations of RpGeometry. For example, changing a facial expression from a frown to a smile, with a hint of a grin. In the process, the base RpGeometry has one or more "delta morph targets" (RpDMorphTarget) (or "deltas" for short) applied to it. The RpDMorphTarget's can overlap and morph any combination of the base RpGeometry's vertex components: positions, normals, pre light colors and texture coordinates.

16.1.2 DMorphing

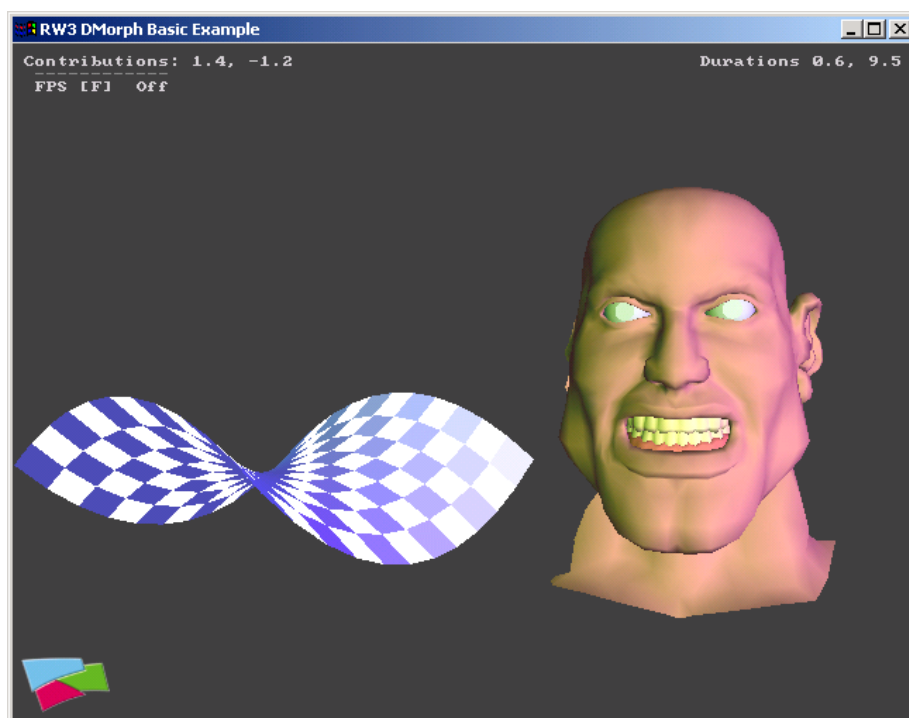
In RenderWare Graphics, DMorphing also differs from normal morphing in the way the targets are stored internally. The base RpGeometry and RpDMorphTarget's are created in the exporter or procedurally – where each is, initially, an absolute and complete model. The RpDMorphTarget's are stored as a plugin extension of the base RpGeometry data, and are compressed by excluding sequences of vertices where the delta is zero. As such it makes it much more efficient in terms of memory usage.

16.1.3 Animation

Although it is possible to directly manipulate the amounts that each RpDMorphTarget is applied to the base RpGeometry, a standard animation system is provided where a series of keyframes can be applied to each RpDMorphTarget and used to animate the system over time.

16.1.4 Examples

The example, found in `examples/dmorph` , will be used in this chapter to illustrate the features provided by the DMorph plugin's API. The example uses two different `RpGeometry` objects to illustrate the two ways in which `RpDMorphTarget` s can be used – a human face which is animated, and a curved surface which can be manipulated – both have a base `RpGeometry` and two `RpDMorphTarget` s.



DMorph Example

16.2 Basic DMorph Usage

Before any of the plugin's features can be used, the plugin should be attached using `RpDMorphPluginAttach()`. Note, DMorphing is fully compatible with the skinning and matfx plugins, but it is not compatible with the (normal) morphing plugin.

In this section, we shall assume that we have a pre-built DMorph example, complete with `RpGeometry`, `RpDMorphTarget` s and an `RpDMorphAnimation` – such as the face in the example.

16.2.1 Loading a pre-built example

It is useful to realize that `RpGeometry` and its `RpDMorphTarget` s are separate from any animation (in fact, DMorphing can be achieved without any explicit animation):

Geometry

A `.DFF` file contains the base `RpGeometry` and all of its associated `RpDMorphTarget` s. `RpClumpStreamRead()` can be used to load this.

Animation

A `.DMA` file contains the details of the frames, and can be loaded using `RpDMorphAnimationRead()` – internally, this opens an `RwStream` on the file, finds the DMorph animation chunk header and then calls `RpDMorphAnimationStreamRead()` before closing the `RwStream`. The animation is stored as an `RpDMorphAnimation`.

Before any DMorphing can take place, the `RpDMorphAnimation` has to be set onto the `RpAtomic` using `RpDMorphAtomicSetAnimation()` (note, this is only applicable where the `RpAtomic` has `RpGeometry` with `RpDMorphTarget` s attached). (`RpDMorphAtomicGetAnimation` can be used to get the animation.)

In the DMorph example, the face is a pre-built example:

```
/*
 * Load DMorphing animation file...
 */

<BaseAnimation> =
    RpDMorphAnimationRead (RWSTRING("./models/face.dma"));
```

16.2.2 Animating

The `RpAtomic` has an animation interpolator which is set to the initial frame in the `RpDMorphAnimation` for each `RpDMorphTarget`, and the interpolation times are set to zero. The DMorph values are set to those at the start of the animation.

The animation can subsequently be advanced during run-time with `RpDMorphAtomicAddTime()`.

In the example, the face is animated over time. In addition to the basic DMorph usage described here, the animation settings of the face can be controlled by the user. (This is discussed further in Section 16.4)

```
/*  
 * Update the animation of the atomic...  
 */  
  
RpDMorphAtomicAddTime (<atomic>, <deltaTime>);
```

16.3 RpGeometry and RpDMorphTargets

16.3.1 RpGeometry

An `RpGeometry` () can be created in the exporter for involved or complex geometries, or procedurally for relatively simple or regular geometries such as cubes, spheres and surfaces, etc. The base `RpGeometry` is created as any normal `RpGeometry` , and is given one morph target using `RpGeometryGetMorphTarget()` (note the difference between “morph target” and “delta morph target” – remember `DMorphing` is not compatible with normal morphing.) Each of the “to-be” deltas are also generated in this way – exactly as the base `RpGeometry` . However, only the base `RpGeometry` should be put in an `RpClump`.

For convenience, `RpDMorphTargetGetBoundSphere()` can be used to get the bounding sphere of the `RpDMorphTarget` . The bounding sphere is returned as if the `RpDMorphTarget` had been fully applied to the base `RpGeometry` .

In the example, the curved surface is generated procedurally. Three similar geometry objects are created all of which, necessarily, have the same number of vertices. The base `RpGeometry` is a flat surface, and the two `RpDMorphTarget` s are curved along one of the x and z axes.

16.3.2 Adding RpDMorphTargets

To attach the deltas, `RpDMorphGeometryCreateDMorphTargets()` is called with the base `RpGeometry` and creates space for a number of `RpDMorphTarget` s. Each target can then be added to the base `RpGeometry` using `RpDMorphGeometryAddDMorphTarget()` . With this function, the delta is assigned an `RpDMorphTarget` index and the vertex data which is to be morphed: vertices, normals, pre light colors and texture coordinates. These also have to be passed in the flag field, which can be logically or'ed together; the flags are: `rpGEOMETRYPOSITIONS`, `rpGEOMETRYNORMALS`, `rpGEOMETRYPRELIT`, `rpGEOMETRYTEXTURED`.

To pass the vertex positions of an `RpGeometry` object, `RpMorphTargetGetVertices()` can be used on `RpGeometryGetMorphTarget()` for the `RpGeometry` object in question. (Likewise, similar functions exist to obtain the normals, pre-light colors and texture coordinates.)

In the example, the surface has vertices and normals:

```
/*  
 * Add DMorph targets to the base surface geometry...  
*/
```

```
RwV3d *verts;
RwV3d *norms;

/* create space for 2 delta morph target */
RpDMorphGeometryCreateDMorphTargets (<BaseGeom>, 2);

verts = RpMorphTargetGetVertices
        ( RpGeometryGetMorphTarget (<DeltaGeom>, 0));

norms = RpMorphTargetGetVertexNormals
        ( RpGeometryGetMorphTarget (<DeltaGeom>, 0));

if (! RpDMorphGeometryAddDMorphTarget (<BaseGeom>, 0,
        verts, norms, NULL, NULL,
        rpGEOMETRYPOSITIONS| rpGEOMETRYNORMALS)
{
    <Error>
}

. . .
```

The flags define which elements should be DMorphed and be queried using `RpDMorphTargetGetFlags()` on an `RpDMorphTarget`, which can be obtained from the base `RpGeometry` with `RpDMorphGeometryGetDMorphTarget()`.

For convenience, `RpDMorphTarget`s can be named using `RpDMorphTargetSetName()` (or they can be named in the exporter). With the face example, labels of “rage” and “smile” could be used. An `RpDMorphTarget` name can then be retrieved later using `RpDMorphTargetGetName()`.

16.3.3 Saving DMorph RpGeometry

Once everything has been generated and attached, we can save the `RpGeometry` with its `RpDMorphTarget`s using `RpClumpStreamWrite()`.

16.3.4 Direct control of DMorph Values

Whether generated procedurally or pre-built, the number of `RpDMorphTarget`s of an `RpGeometry` can be obtained using `RpDMorphGeometryGetNumDMorphTargets()`. This is useful when used in conjunction with `RpDMorphAtomicGetDMorphValues()` for directly controlling the DMorph values. The latter function returns a pointer to an array of `RwReals` that are the contributions that each `RpDMorphTarget` has on the base `RpGeometry` – and thus the values can be overwritten directly to alter the DMorphed `RpGeometry`. (It may also be done in combination with a standard DMorph animation, see Section 16.4.)

The contributions that each `RpDMorphTarget` applies is normally in the range [0, 1], where a value of zero means that no contribution is being applied, while a value of one means that the entire contribution is being applied. However, the value may be set out of this range, including negative values.

Note, you need to call `RpDMorphAtomicInititalize()` to overload the `RpAtomic` so that it can be DMorphed – otherwise the object will remain rigid. However, this function is called automatically when DMorphed `RpGeometry` is loaded.

In the example, the DMorph values can be controlled directly by the user to alter the shape of the curved surface:

```
/*
 * Alters the surface contributions...
 */

RwInt32 max = RpDMorphGeometryGetNumDMorphTargets (
    RpAtomicGetGeometry(<atomic>));

RwReal *dlist;
RwInt32 i;

dlist = RpDMorphAtomicGetDMorphValues (<atomic>);

for (i=0; i<max; i++)
{
    dlist[i] = <contrib_array>[i];
}
```

16.3.5 Transforming RpGeometry with RpDMorphTargets Attached

Transforming the `RwFrame` of the base `RpGeometry` will successfully transform the `RpGeometry` and its associated `RpDMorphTargets`.

However, if `RpGeometryTransform()` is called (to transform the `RpGeometry`'s `RpMorphTarget` vertices, see the Dynamic Models chapter), then `RpDMorphGeometryTransformDMorphTargets` can be used to apply the specified transformation matrix equally to all `RpDMorphTarget`s defined on the base `RpGeometry` transforming both the vertex position deltas and vertex normal deltas.

16.3.6 Destroying RpDMorphTargets

An `RpDMorphTarget` can be removed from an `RpGeometry` object using `RpDMorphGeometryRemoveDMorphTarget()` (freeing up the data created with `RpDMorphCreateDMorphTargets()` and creating room for a new `RpDMorphTarget` to be added). Note, if the target was applying a contribution to the base geometry when it was removed, that influence will remain intact. To remedy this, the `DMorph` value should be directly set to zero before removal.

All `RpDMorphTarget` s can be destroyed with `RpDMorphGeometryDestroyDMorphTargets()` .

16.4 Animation

16.4.1 Creating Frames

An animation takes "delta morph animation " the form of a series of frames that are associated with each `RpDMorphTarget` . An `RpDMorphTarget` can have an `RpDMorphAnimation` created with `RpDMorphAnimationCreate()` . For each animation one or more keyframe sequences can be created using `RpDMorphAnimationCreateFrames()` . The function must be called for every `RpDMorphTarget` that is to be controlled by the `RpDMorphAnimation` . (Some sequences may be left absent for `RpDMorphTarget` s which are unused or which are to be procedurally controlled externally.)

Each frame can then be set using `RpDMorphAnimationFrameSet()` whose arguments are:

<code>Anim</code>	A pointer to the <code>RpDMorphAnimation</code> object
<code>DMorphTargetIndex</code>	The index to identify the <code>RpDMorphTarget</code>
<code>FrameIndex</code>	The index to identify the frame
<code>StartValue</code>	The contribution of the delta to the base <code>RpGeometry</code> at the start of the frame
<code>EndValue</code>	The contribution of the delta to the base <code>RpGeometry</code> at the end of the frame
<code>Duration</code>	The duration of the frame in seconds
<code>NextFrame</code>	The index of the next frame

Alternatively, the last four arguments can be set individually with:
`RpDMorphAnimationFrameSetStartValue()` ,
`RpDMorphAnimationFrameSetEndValue()` ,
`RpDMorphAnimationFrameSetDuration()` and
`RpDMorphAnimationFrameSetNext()` , respectively. The values can be obtained using the "get" version of these functions.

Finally, the animation is set onto the `RpAtomic` using `RpDMorphAtomicSetAnimation()` .

In the example, the face has a number of frames per `RpDMorphTarget` . The user can increase or decrease the duration of the frames as a group, and can control each `RpDMorphTarget` independently:

```
/*  
 * Change durations of the face animation...  
*/  
  
RwUInt32 frames;  
RwUInt32 i;
```

```

frames = RpDMorphAnimationGetNumFrames (<BaseAnimation>, 0);
for (i=0; i<frames; i++)
{
    RpDMorphAnimationFrameSetDuration (
        FaceBaseAnimation, 0, i, <duration>);
}

```

16.4.2 Saving Animations

Once everything has been setup, an `RpDMorphAnimation` can be saved using `RpDMorphAnimationWrite()`. (This internally calls `RpDMorphAnimationStreamGetSize()` and `RpDMorphAnimationStreamWrite()`.)

16.4.3 Editing and Querying Frame Sequences

`RpDMorphAtomicSetAnimFrame` will set the specified `RpDMorphTarget` to the start of a particular frame in an `RpDMorphAnimation`. (A value of `rpDMORPHNULLFRAME` may be specified for the frame index which effectively disconnects a particular `DMorphTarget` from the `RpDMorphAnimation`.)

`RpDMorphAtomicGetAnimFrameTime()` can be used to get the interpolated time within the current animation frame of the specified `RpDMorphTarget` – zero at the start of the frame and equal to the frame duration at the end. Similarly, `RpDMorphAtomicSetAnimFrameTime()` is used to set the interpolation time within the current animation frame for a particular `RpDMorphTarget`.

`RpDMorphAtomicGetAnimTime()` is used to obtain the total amount of time added to the animation of a delta morph atomic. (It is impossible to set the absolute animation time directly – but this can be achieved using `RpDMorphAtomicSetAnimation()` and then adding the appropriate time with `RpDMorphAtomicAddTime()`.)

16.4.4 Loop Callbacks

`RpDMorphAtomicSetAnimLoopCallBack()` is used to set an `RpAtomic` callback that will be called whenever an `RpDMorphAnimation` loops during `RpDMorphAtomicAddTime()`. The function can be retrieved using `RpDMorphAtomicGetAnimLoopCallBack()`. (There is no default callback.)

16.4.5 Running an Animation

`RpDMorphAtomicAddTime()` is used to advance the animation of a `DMorph RpAtomic` by the given amount of time (an animation must have already been attached with `RpDMorphAtomicSetAnimation()`).

It is not possible to play an animation backwards, and adding negative time will produce invalid results. Note that if the animation loops, the time returned by this function does not reset to zero. It is the total time added to the animation including loops.

16.4.6 Destroying Frames

`RpDMorphAnimationDestroyFrames()` destroys the keyframe sequence in an `RpDMorphAnimation` corresponding to a particular `RpDMorphTarget` .

`RpDMorphAnimationDestroy()` destroys an `RpDMorphAnimation` and any keyframe sequences it contains.

16.5 Summary

16.5.1 Delta Morphing

Delta morphing has a number of targets (`RpDMorphTarget`) that can be applied to a base geometry (`RpGeometry`).

Each target contributes to the overall `RpGeometry` .

Targets can overlap.

It is possible to DMorph vertex positions, normals, pre light colors, and texture coordinates.

The example, `DMorph`, demonstrates basic usage, `RpGeometry` & `RpDMorphTarget` and `RpDMorphAnimation` .

16.5.2 Basic Usage

Delta morphing can be considered two-part. The `RpGeometry` and `RpDMorphTarget` s, and the `RpDMorphAnimation` .

With a pre-built model:

It can be loaded with `RpClumpStreamRead()` .

Its animation is loaded using `RpDMorphAnimationRead()` .

In general:

`RpDMorphAtomicSetAnimation()` is used to set the `RpDMorphAnimation` onto the atomic.

The `RpDMorphAnimation` can be run using `RpDMorphAtomicAddTime()` at run-time.

16.5.3 RpGeometry and RpDMorphTargets

For the `RpDMorphTarget` :

`RpDMorphGeometryCreateDMorphTargets()` creates space for a number of `RpDMorphTargets` .

`RpDMorphTargets` are added using `RpDMorphGeometryAddDMorphTarget()` .

DMorph values can be controlled directly using `RpDMorphAtomicGetDMorphValues()` which returns the array of contributions from each `RpDMorphTarget` .

16.5.4 RpDMorphAnimation

An RpDMorphAnimation is created using RpDMorphAnimationCreate() .

Frames can be added to each RpDMorphAnimation with
RpDMorphAnimationCreateFrames() .

Frames are set using RpDMorphAnimationFrameSet() .

Finally, RpDMorphAtomicSetAnimation() is used to set-up the whole system.

Part D

Special Effects Libraries

Chapter 17

The Material Effects Plugin

17.1 Introduction

The Material Effects plugin, RpMatFX, provides a set of off-the-shelf material effects which can be applied to materials used by atomics and world sectors.

This chapter explains the features provided by this plugin and how to use them.

17.1.1 How RpMatFX Works

The RpMatFX plugin exposes a high-level API to a set of off-the-shelf effects pipelines. The high-level API hides the complexities of setting up the effects, which differs widely from platform to platform.

17.1.2 RpMatFX Features

The RpMatFX plugin supports four effects:

- Environment mapping

- Bump mapping

- Environment & bump mapping

- Dual-pass texture mapping

These effects can be applied to a material used by either an atomic or world sector object. Different materials can have different effects enabled, so an atomic might, for example, be rendered with materials supporting both environment mapping and dual-pass texture mapping.

Material Effects & Modeling Packages

Artists usually specify the effect(s) required for a specific model during the export stage. The RenderWare Graphics modeling package exporters support this process—detailed documentation can be found in the particular Artists Guide relevant to the modeling package.

17.2 Using Material Effects

The Material Effects plugin works on `RpMaterial` objects rendered within atomics or world sectors.

The procedure for using `RpMatFX` is a four-step process:

1. Select the desired effect and apply it to the material
2. Initialize any additional data for the effect
3. Enable the `RpMatFX` renderer for the atomic or world sector which uses the material
4. Render the scene

The following sections will cover each step in more detail.

17.2.1 Selecting The Effect

The `RpMatFX` API exposes the `RpMatFXMaterialSetEffects()` function to select the required effect. This must be the first function called when setting up an effect on a material.

The `RpMatFXMaterialSetEffects()` function requires a pointer to the `RpMaterial` object that is to have the effect applied to it, as well as a flag which defines the effect to apply. The flag settings are listed in the table below:

FLAG	EFFECT
<code>rpMATFXEFFECTBUMPMAP</code>	Enables bump mapping on the chosen material.
<code>rpMATFXEFFECTENVMAP</code>	Enables environment mapping on the chosen material.
<code>rpMATFXEFFECTBUMPENVMAP</code>	Enables both bump and environment mapping on the chosen material.
<code>rpMATFXEFFECTDUAL</code>	Enables dual pass texturing

17.2.2 Initializing Effect Data

Once an effect has been chosen, any additional data needed to implement that effect—additional texture maps, bump maps, etc.—need to be set up so the effect can be rendered.

The initialization steps vary depending on the effect selected, so this section looks at each of the effects in turn.

The effects supported are:

Bump mapping

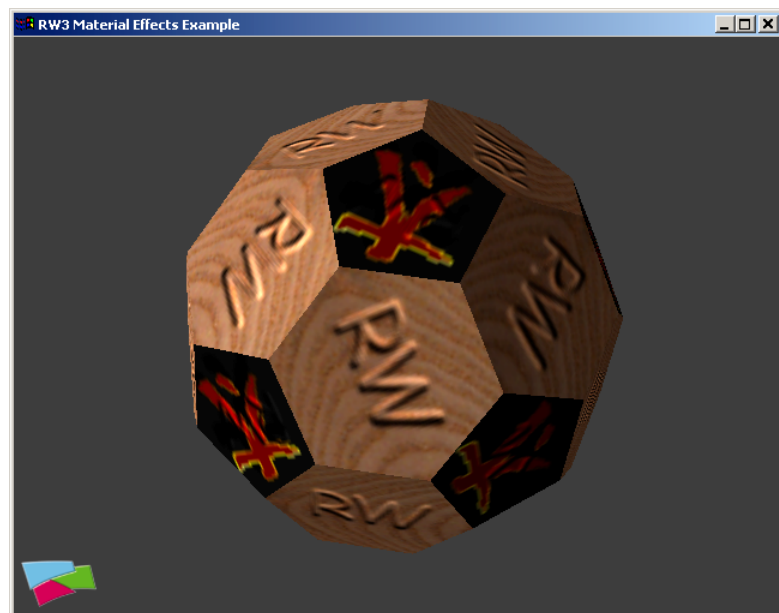
Environment mapping

Bump & environment mapping

Dual-pass texture mapping

Bump Mapping

The bump mapping effect renders a second bump texture over a base texture such that the illusion of a bumpy surface is created. An example is shown in the screenshot below.



A bump mapped model

Each bump mapped material requires the following properties to be initialized:

- A bump map texture defining the 'bumpiness'

- The definition of the light direction for the bump map

- The bump map coefficient, which defines the intensity of the bump mapping

The `RpMatFXMaterialSetupBumpMap()` function is used to set up the bump mapping effect's properties for each material in one call. In addition to this setup function, the properties also have individual access functions, which are useful if you need to modify a material effect property after it has already been initialized:

Setting Bump Mapping Properties

The functions described below are used to set the bump map properties for a material.

`RpMatFXMaterialSetBumpMapTexture()` – sets the bump map texture;

`RpMatFXMaterialSetBumpMapFrame()` – sets the bump map lighting direction, (represented by an `RwFrame` object). If this property is left undefined, the frame is derived from the current camera's at vector;

`RpMatFXMaterialSetBumpMapCoefficient()` – sets the bump map's coefficient.

Retrieving Bump Mapping Properties

The functions described below are used to retrieve the bump map properties for a material.

`RpMatFXMaterialGetBumpMapTexture()` – returns the material's current bump map texture;

`RpMatFXMaterialGetBumpMapFrame()` – returns the material's current bump map lighting direction as an `RwFrame` object;

`RpMatFXMaterialGetBumpMapCoefficient()` – returns the material's bump map coefficient.

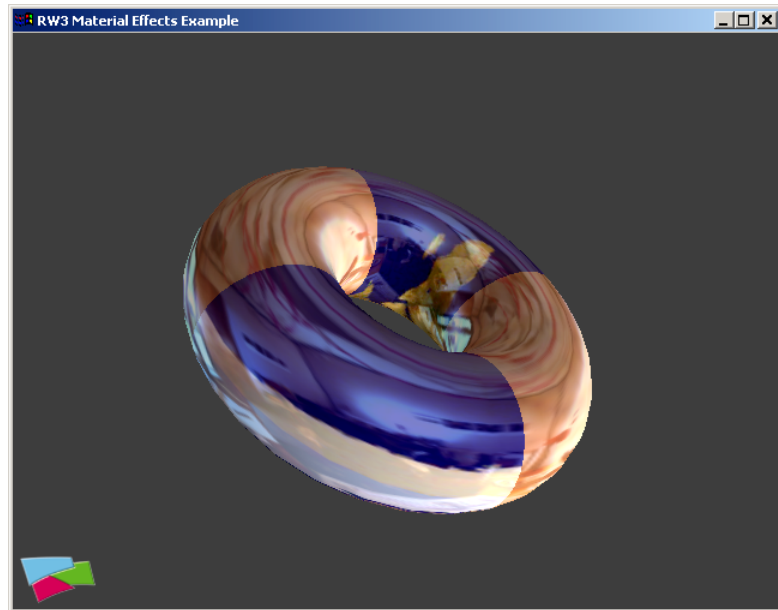
Bumped Textures

When a bump map texture is set, the `RpMatFX` plugin integrates it into the alpha channel of the base texture. The resulting, combined, texture is called a `bumped` texture and a pointer to it is obtained with a call to `RpMatFXMaterialGetBumpedTexture()` .

Environment Mapping

The environment mapping effect creates the illusion of a reflective surface by mapping an environment texture—a texture containing an image of the material's surroundings—onto the material.

Variations on the effect can be created by using different images for the environment texture. For example, a texture that contains only highlights can be used to create the effect of a glossy surface.



An environment mapped object

Each environment mapped material requires the following properties to be initialized:

- The environment map texture

- An `RwFrame` which defines the environment map's projection. If this is not defined, a default frame object is generated which always faces the current camera

- A flag defining whether the frame buffer's alpha channel should be used when applying the environment map

- An environment map coefficient, which defines how reflective the material is, i.e. the intensity of the environment map

As with the bump mapping effect, each property also supports individual access functions.

Setting Environment Mapping Properties

The functions described below are used to set the environment map properties for a material.

- `RpMatFXMaterialSetEnvMapTexture()` – sets the environment map texture
- `RpMatFXMaterialSetEnvMapFrame()` – sets the environment mapping projection (an `RwFrame` object)
- `RpMatFXMaterialSetEnvMapFrameBufferAlpha()` – a boolean value, which should be set to `TRUE` if the frame buffer's alpha channel is to be used
- `RpMatFXMaterialSetEnvMapCoefficient()` – sets the environment map coefficient

Retrieving Environment Mapping Properties

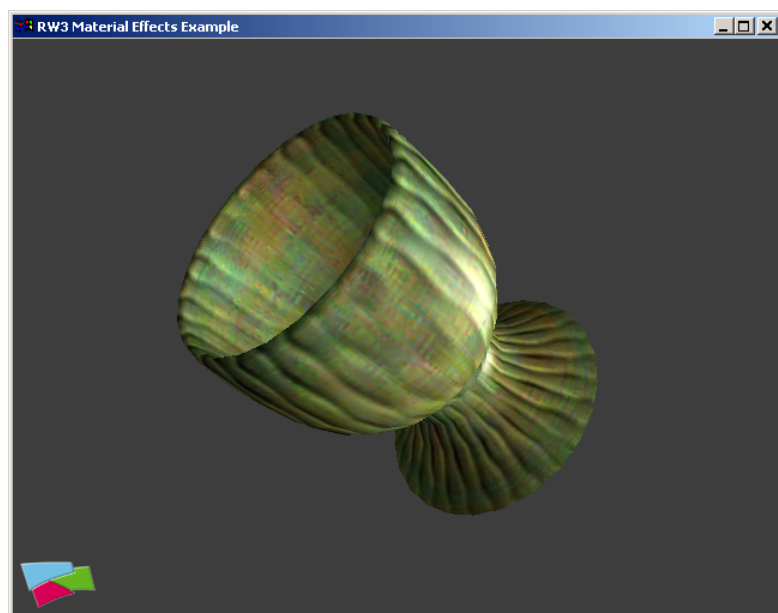
The functions described below are used to retrieve the environment map properties for a material.

- `RpMatFXMaterialGetEnvMapTexture()` – returns the environment map texture
- `RpMatFXMaterialGetEnvMapFrame()` – returns the environment mapping projection
- `RpMatFXMaterialGetEnvMapFrameBufferAlpha()` – returns `TRUE` if the frame buffer's alpha channel will be used
- `RpMatFXMaterialGetEnvMapCoefficient()` – returns the environment map coefficient

Bump & Environment Mapping

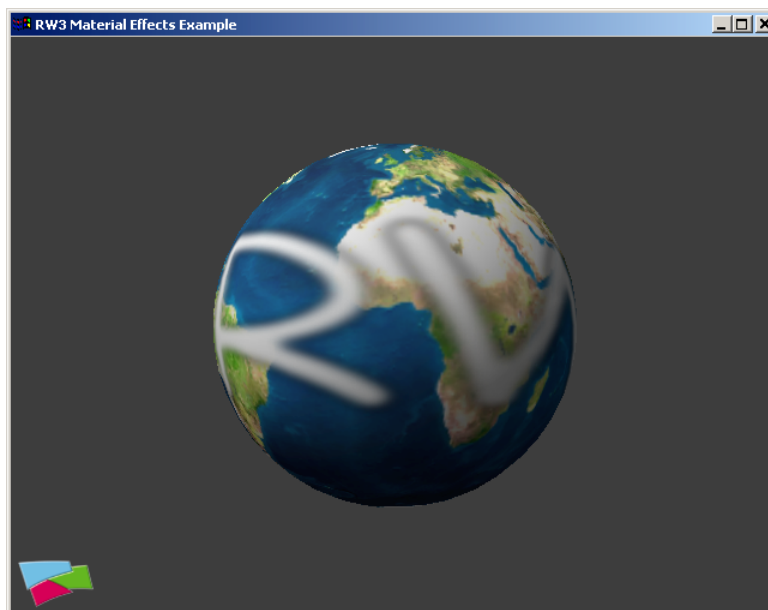
This effect combines both bump mapping and environment mapping effects, covered in detail above. This effect is achieved by combining both bump and environment mapping effects. The property setup process is performed by calling both the `RpMatFXMaterialSetupBumpMap()` and `RpMatFXMaterialSetupEnvMap()` functions.

The property access functions are also identical to those described earlier in the bump mapping and environment mapping sections.



An object with both environment and bump mapping

Dual-Pass Texture Mapping



An object with dual-pass texturing.
(The overlaid 'RW' text is the second texture.)

This effect works by combining the material's own texture with a second texture according to specified combination flags.

The setup function is `RpMatFXMaterialSetupDualTexture()` and sets the following properties:

The second texture (the first is defined by the `RpMaterial` object)

The blend function for the source data

The blend function for the target data

Blend Functions

The two blend functions determine how the two textures are blended with the data in the frame buffer. They are defined by the `RwBlendFunction` enumeration. A brief description of each flag follows—see the `Blending` section in the `Immediate Modes` chapter for a complete description of the blending system.

`rwBLENDNABLEND` "Not A Blend" – no blending is performed

`rwBLENDZERO` RGBA channels set to zero

`rwBLENDONE` RGBA channels are set to 1

`rwBLENDSRCCOLOR` Source RGBA only

`rwBLENDINVSRCCOLOR` Inverse of source RGBA only

rwBLENDSRCALPHA Source alpha only on all channels

rwBLENDINVSRCALPHA Inverse source alpha only on all channels

rwBLENDDESTALPHA Destination alpha only on all channels

rwBLENDINVDESTALPHA Inverse destination alpha only on all channels

rwBLENDDESTCOLOR Destination RGBA values only

rwBLENDINVDESTCOLOR Inverse destination RGBA only

rwBLENDSRCALPHASAT Source alpha (saturated)

Each dual-pass effect property has its individual access functions.

Setting dual-pass effect properties

The functions described below are used to set the dual-pass effect properties for a material.

RpMatFXMaterialSetDualTexture() – sets the second texture. The first texture is the one already contained within the RpMaterial object.

RpMatFXMaterialSetDualBlendModes() – sets both blending functions.

Retrieving dual-pass effect properties

The functions described below are used to set the dual-pass effect properties for a material.

RpMatFXMaterialGetDualTexture() – returns a pointer to the second texture.

RpMatFXMaterialGetDualBlendModes() – returns the settings of the two blending functions.

17.2.3 Enabling the Effects Renderer

Once the data for the effect has been initialized for a material, the RpMatFX renderer needs to be enabled for the atomic or world sector that contains the material.

If this operation is not performed, the additional effects data will be ignored and the material rendered using only the basic RpMaterial data.

Enabling the effects renderer on an atomic or world sector only needs to be performed once, regardless of how many RpMatFX-extended materials it contains.

Enabling Effects on an Atomic

To enable material effects for an atomic which contains an RpMatFX-extended material, call: `RpMatFXAtomicEnableEffects()` , passing the relevant atomic as a parameter.

Enabling Effects on a World Sector

To enable material effects for a world sector use:
`RpMatFXWorldSectorEnableEffects()` , passing the relevant atomic as a parameter.

Rendering

Once the effects have been enabled on the relevant atomics and world sectors, rendering of the affected materials is performed automatically.

The Material Effects plugin's own renderer detects materials with RpMatFX data as they pass through RenderWare Graphics' rendering pipeline. When one is found, the plugin temporarily substitutes its own rendering pipeline to render these materials, reverting to the normal rendering pipeline for normal materials.

17.3 Examples

The following example shows how to set up an atomic containing a bump mapped material, then render it. Only relevant code fragments are shown.

A complete RpMatFX example is provided with the SDK's examples folder, named matfx1 .

17.3.1 A Bump Mapping Example

Preparation

As was discussed in section 1.2.2, each bump mapped material requires the following properties to be set:

- A bump map texture defining the 'bumpiness';

- A definition of the light direction for the bump map;

- A bump map coefficient.

The `RpMatFXMaterialSetupBumpMap()` function is used to set the required data for each material.

A number of objects are involved in this example. The first is the atomic object which contains the model with the material which will contain our bump map.

```
RpAtomic *myAtomic;
```

Next, there's the material itself and the bump map texture which will be applied to it.

```
RpMaterial *myMaterial;
```

```
RwTexture *bumpTexture;
```

In addition, the bump map texture will also need a frame. This frame determines the direction of the bump map's lighting. This lighting gives us the effect of a bumpy surface, even though the surface is really flat.

```
RwFrame *bumpLighting; /* bump map lighting direction */
```

It should be noted that this light, which behaves similarly to directional lighting, is not a real light: it only affects the bump map on this particular material.

Other lights that impinge on this object will light the model as usual, but you will need to modify the bump map lighting to match in order to maintain the illusion.

Finally, we need a bump coefficient, which is defined as an `RwReal`, (defined here as 0.77, but it could be any value). This determines how bumpy the surface will appear to be. A low value gives only a slightly bumpy surface whereas a larger value produces a bumpier effect.

```
RwReal bumpCoefficient = 0.77; /* bump coefficient */
```

For the purpose of this example, these objects are assumed to have been already initialized with valid data.

Initializing the Material Effect

With the objects defined and initialized, the first step in the initialization process is to set the desired effect for the material.

```
RpMatFXMaterialSetEffects( myMaterial, rpMATFXEFFECTBUMPMAP );
```

This tags the material as a bump mapped material, but we still need to initialize the necessary data the bump map.

```
RpMatFXMaterialSetupBumpMap(    myMaterial,
                                bumpTexture,
                                bumpLighting,
                                bumpCoefficient );
```

Enabling Effects on the Atomic

For the purposes of this example, the `RpMaterial` object represented by `myMaterial` is assumed to be already contained within the atomic named `myAtomic`.

In order for the bump mapped material to render with the `RpMatFX` renderer, the atomic it is contained within must be enabled for material effects rendering:

```
RpMatFXAtomicEnableEffects( myAtomic );
```

Rendering the Effect

The Material Effects plugin hooks into the RenderWare Graphics rendering engine, so rendering of the atomic containing our bump mapped material can be performed using either:

```
RpAtomicRender( myAtomic );
```

or, if the atomic has been added to an `RpWorld` object, (named `myWorld`, in the example below) by a call to:

```
RpWorldRender( myWorld );
```

This completes the example.

17.4 Summary

The Material Effects plugin, RpMatFX, provides a set of off-the-shelf material effects which can be applied to materials used by atomics and world sectors.

17.4.1 Supported Effects

The RpMatFX plugin supports four effects:

- Environment mapping

- Bump mapping

- Environment & bump mapping

- Dual-pass texture mapping

17.4.2 Extended Objects

Materials

The Material Effects plugin works by extending RpMaterial objects with necessary data for its supported effects.

The data must be set up prior to rendering, using the appropriate RpMatFXEffectSetup...() function.

Atomics & World Sectors

RpMatFX hooks into RpAtomic and RpWorldSector object rendering, so a material must be used in either—or both—objects in order for it to be rendered.

The RpMatFX plugin will only hook into the renderer for an atomic or world sector by a call to either RpMatFXEnableAtomicEffects() or RpMatFXEnableWorldSectorEffects(), respectively. These functions enable the effects on the objects passed.

Chapter 18

PTank

18.1 Introduction

This Chapter introduces the RpPTank plugin and the concepts of the particle and the particle tank. It describes the way they save processing time by side-stepping the usual 3D processes of RenderWare Graphics, and it explains how to use them.

18.1.1 What is a Particle?

In RenderWare Graphics a particle is a 2D shape.

It may have a single color that may have degrees of transparency. It may have an image or texture applied to it by UV coordinates.

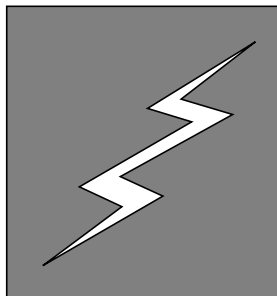
It is very similar to a sprite in early computer games.

A particle can be scaled up or down; it can be moved, even animated.

It does not exist as a separate data structure; it is integral to the particle tank.

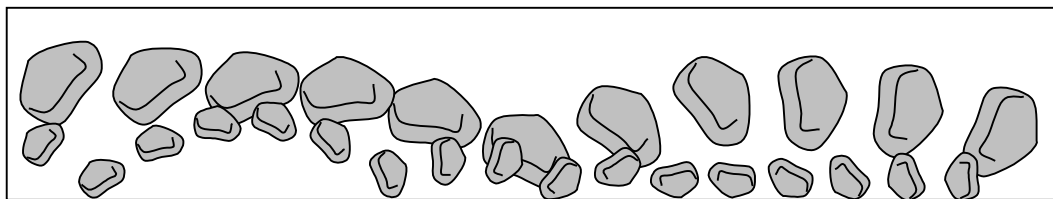
18.1.2 What Are Particles Used For?

It is often unnecessary to use RenderWare Graphics' usual 3D graphics capability. The memory required for a geometry, and the processing time required for each frame can be wasted on simple, dramatic or fleeting effects, and particles are good when transient, fast moving, small or flat images are required.



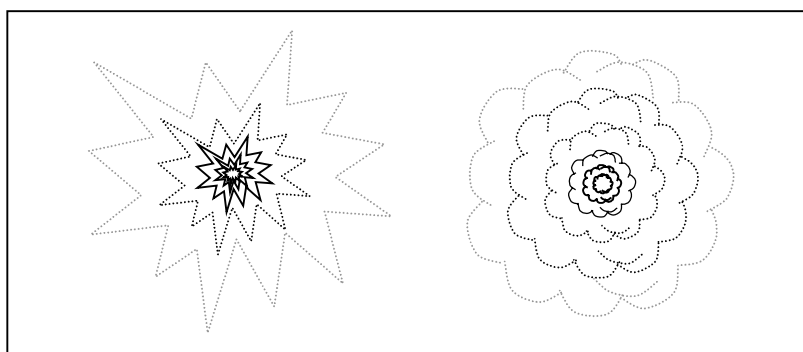
Particles are good for transient effects

An RpPTank particle can represent a moving rock. Particles can be scaled, so the image can get bigger as the rock approaches. The image can rotate. This is enough to convey rocks tumbling down a cliff or missiles hurled through the air.



One particle in two sizes represents two tumbling rocks

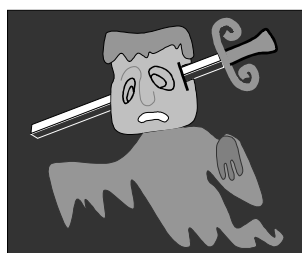
In some games, objects explode in a cloud. This sudden event requires only one image expanded rapidly. RpPTank particles can reduce their opacity (their alpha value) to disappear like a cloud and they can be scaled and rendered very quickly.



A single scaled particle can represent a crash or an explosion adequately

Flames can be conveyed by a series of semi-transparent images in front of the burning object. Particles provide the simple 2D animation that this effect needs, and 3D processing is not necessary. But particles can be displayed behind as well as in front of the burning object, so they can suggest 3D flames very effectively.

Tiny objects like feathers, snowflakes and sparkling highlights can be displayed, moved and replaced or removed very efficiently as particles, without any of the complexity of 3-D calculations.



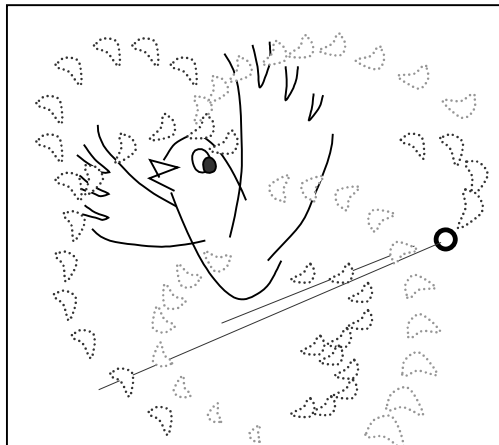
Particles are suited to cartoon-like transparent images

Simple ghosts, apparitions and specters require images that are bold but transparent, scaled, moving, and front-facing. If these images do not need to be articulated they can be rendered easily by particles.

The implementation of PTank allows the developer to store and animate data conveniently in simple ways. Controlling the movements of particles through space is a task for another plugin, like RpPrtStd .

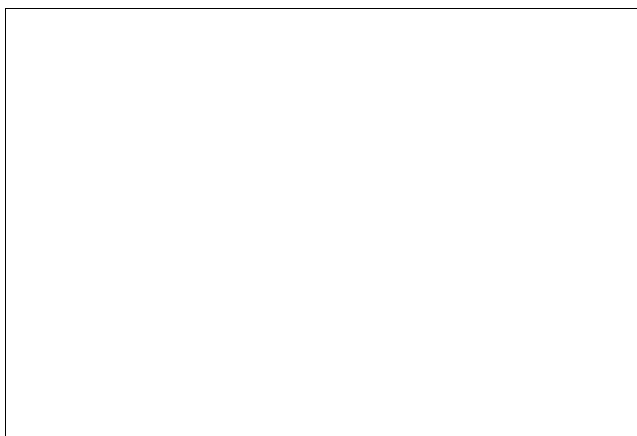
18.1.3 What Is the Particle Tank?

The particle tank is a collection of particles. The word "tank" is used to suggest a container for particles.



Particles can be animated to represent feathers

The format of the particle tank or RpPTank is flexible. It can be organized as a structure or an array format and its data contents vary according to the image it conveys. Similar particles, like a flurry of feathers, or a sequence of transparent disks to represent smoke might be stored in the same RpPTank.



Transparent disk particles can represent smoke

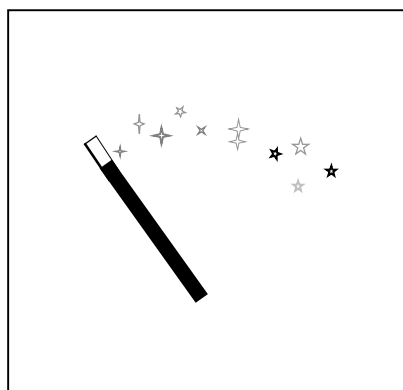
Particle tanks can hold particles of very different types, and several contrasting particle tanks can be active at the same time.

18.1.4 What Particles Are Not

Particles not sophisticated or complicated. They are a simpler form of representation than RenderWare Graphics provides for 3D worlds and animated objects.

A particle is not three dimensional. It is a flat image like a sprite. So, by default, it is rendered as if it is parallel to the near Z plane, facing the camera. As a result, if the camera moves, the particle appears to turn to face the camera. Alternatively it must appear to be the same from all view points, like a glowing sphere.

The RpPTank plugin could be used to manipulate letters and titles and move them around the screen, but the Rt2d toolkit manipulates letters is designed to achieve these effects more easily.



Particles are suited to transient effects, like sparkle

Particles are also known for realistically representing galaxies, waterfalls, trees, vegetation and foliage, roman candles, rockets and other fireworks, realistic cloud formations, crowds and snow storms. These require not only particles, but a means to control their movements and interactions, and that is beyond the scope of RpPTank.

18.1.5 Other Documents

Little background knowledge is required for this topic.

The API Reference gives a description of this plugin, each of its data structures and functions.

Other concepts used in this Chapter are described elsewhere. Textures and their coordinates, colors and plotting modes are covered in The Material Effects Plugin , and Immediate Mode chapters.

RpPTank interfaces with routines that differ widely across platforms. Be sure to consult the appropriate platform-dependent API Reference.

18.2 The Main Concepts

The concepts of a particle and the way it is stored in a PTank are central to this plugin. To use it, the developer also needs to understand the descriptor of the PTank's structure and its locking mechanism. They are described here.

18.2.1 The Particle

Particles are only defined in the particle tank, and there are two formats for the particle tank and several optional data fields that define it.

The definition structure, `RpPTankFormatDescriptor` holds a flag field. Two flags represent the PTank's data "organization". The other flags represent specific items of data. The preprocessor "PTank Data flag" values that follow represent the items of data that may be included. These values are passed to the `RpPTankAtomicCreate()` function to create the respective arrays.

Many of the following flags that describe a particle have an almost identical name containing the letter "L" for "lock" (`rpPTANKLFLAG`) in place of "D" for "data" (`rpPTANKDFLAG`). This similar set of flags is used with `RpPTankAtomicLock()` to lock, read and write the arrays created by the `RpPTankAtomicCreate()` function. The corresponding flags for locking and for creation are very closely related, so they are described together in the rest of this section on The Particle .

Flags for Spatial Description

`rpPTANKDFLAGPOSITION` reserves space for a 3D point that will define the position of each particle. `rpPTANKLFLAGPOSITION` is used to lock, read and write it. The matrix value also contains position data, so this flag is not compatible with `rpPTANKDFLAGMATRIX`. If a particle has no position it must have a matrix to hold the equivalent position data and the debug version will assert if neither is present.

`rpPTANKDFLAGUSECENTER` reserves space for vectors to position and rotate the particle by the center point specified by the function `RpPTankAtomicSetCenter()` rather than its default center or origin. (There is a shared value and there is no equivalent flag to lock, read and write shared values.)

`rpPTANKDFLAGMATRIX` reserves space for a 3D `RwMatrix` to orient the plane of each particle. `rpPTANKLFLAGMATRIX` is used to lock, read and write it. By default the particle is plotted straight onto the view. This has the effect that it is facing the camera. (If there is more than one camera, it is facing every camera from every angle at the same time.) Setting the values of this matrix allows the particle to be reoriented, for example, like a door swinging on hinges. The `RwMatrix` contains position data, so it is not compatible with `rpPTANKDFLAGPOSITION` and it contains dimension data, so it is not compatible with `rpPTANKDFLAGSIZE`.

`rpPTANKDFLAGCNSMATRIX` reserves space for only a single shared or 'constant' 3D matrix to orient all the particles. (There is no equivalent flag to lock, read and write a constant.)

`rpPTANKDFLAGSIZE` reserves space for a 2D vector that stores the dimensions of the rectangle for each particle. `rpPTANKLFLAGSIZE` is used to lock, read and write it. In essence, a particle is rectangular because it is defined in two dimensions. Applying other effects, in particular, applying an `RwTexture` can give it the appearance of any 2D shape within its rectangle.

`rpPTANKDFLAG2DROTATE` reserves space for a real value to express the degree of 2D rotation, of the particle from its default orientation, in radians, clockwise, from $-\pi$ to π . `rpPTANKLFLAG2DROTATE` is used to lock, read and write the rotation value.

`rpPTANKDFLAGCNS2DROTATE` reserves space for only a single shared or "constant", real variable by which all the particles will be rotated. (There is no equivalent flag to lock, read and write a constant.)

`rpPTANKDFLAGNORMAL` reserves space for a normal 3D vector. `rpPTANKLFLAGNORMAL` is used to lock, read and write it.

`rpPTANKDFLAGCNSNORMAL` reserves space for only a single shared or "constant" 3D normal vector for all the particles. (There is no equivalent flag to lock, read and write a constant.)

Color Flags

`rpPTANKDFLAGCOLOR` reserves space for an `rgba` value for each particle. `rpPTANKLFLAGCOLOR` is used to lock, read and write it. The effect of the `rgba` value is subject to the plotting mode. This flag operates differently from the others in the PTank because, by default, a shared color is created with the PTank. The `rpPTANKDFLAGCOLOR` flag overrides the default behavior and creates individual values in place of the shared one.

`rpPTANKDFLAGVTXCOLOR` reserves space for an `rgba` color for each vertex. `rpPTANKLFLAGVTXCOLOR` is used to lock, read and write it.

rpPTANKDFLAGCNSVTXCOLORReserves space for only a single shared or "constant" rgba variable for all the particles. (There is no equivalent flag to lock, read and write a constant.)

Texture Coordinate Flags

rpPTANKDFLAGVTX2TEXCOORDSReserves space for two coordinates representing the top left and bottom right texture coordinates. **rpPTANKLFLAGVTX2TEXCOORDS** is used to lock, read and write them.

rpPTANKDFLAGCNSVTX2TEXCOORDSReserves space for only a single pair of shared or "constant" vertices for the top left and bottom right texture coordinates. (There is no equivalent flag to lock, read and write a constant.)

rpPTANKDFLAGVTX4TEXCOORDSReserves space for four texture UVs corresponding to the four corners of the quadrilateral defined by the particle. **rpPTANKLFLAGVTX4TEXCOORDS** is used to lock read and write it.

rpPTANKDFLAGCNSVTX4TEXCOORDSReserves space for a single group of four shared or "constant" UV coordinates that apply to all the particles in the PTank. (There is no equivalent flag to lock, read and write a constant.)

Organization Flags

rpPTANKDFLAGARRAYReserves space for a flag to indicate that the PTank's organization is an array of structures. The flag exists in all PTanks, so there is no need to reserve space for it, and there is no equivalent flag to lock, read or write it. This setting is incompatible with that below, so both flags may not be set at the same time. But if neither is set **RpPTank** decides which organization to use, depending upon the current platform.

rpPTANKDFLAGSTRUCTUREIndicates that the PTank's organization is a structure of arrays. The flag exists in all PTanks, so there is no need to reserve space for it, and there is no equivalent flag to lock, read or write it. This setting is incompatible with that above, so both flags may not be set at the same time. But if neither is set **RpPTank** decides which organization to use, depending upon the current platform.

Several of these flags are incompatible. There follows a summary of groups of flags that are incompatible, and it is helpful to consider why they are incompatible. The reasons are all covered above, and the distinction between the shared, CNSvalues and the individual values is explained later under this heading on The Particle .

These flags and flag-pairs are mutually incompatible:

rpPTANKDFLAGCNSVTX2TEXCOORDS

rpPTANKDFLAGVTX2TEXCOORDS

rpPTANKLFLAGVTX2TEXCOORDS

rpPTANKDFLAGCNSVTX4TEXCOORDS

rpPTANKDFLAGVTX4TEXCOORDS

rpPTANKLFLAGVTX4TEXCOORDS

These flags and flag-pairs are mutually incompatible:

rpPTANKDFLAGCOLOR

rpPTANKLFLAGVTXCOLOR

rpPTANKDFLAGVTXCOLOR

rpPTANKLFLAGVTXCOLOR

rpPTANKDFLAGCNSVTXCOLOR

The flags and flag-pairs opposite each other in these two columns are incompatible:

rpPTANKDFLAGARRAY

rpPTANKDFLAGSTRUCTURE

rpPTANKDFLAGNORMAL

rpPTANKLFLAGNORMAL

rpPTANKDFLAGCNSNORMAL

rpPTANKLFLAGCNSNORMAL

rpPTANKDFLAG2ROTATE

rpPTANKLFLAG2ROTATE

rpPTANKDFLAGCNS2ROTATE

rpPTANKLFLAGCNS2ROTATE

rpPTANKDFLAGMATRIX

rpPTANKLFLAGMATRIX

rpPTANKDFLAGPOSITION

rpPTANKLFLAGPOSITION

rpPTANKDFLAGMATRIX

rpPTANKLFLAGMATRIX

rpPTANKDFLAGSIZE

rpPTANKLFLAGSIZE

This flag alone is compatible with all others:

rpPTANKDFLAGUSECENTER

In the debug version, incompatible flag settings will cause an assert.

A further distinction between the flags used in creating a PTank is that one value, "position", always applies to all particles, some values are shared between all particles and some can be either shared or independent. The table below summarizes these differences.

INDEPENDENT VALUES	VALUES THAT MAY BE EITHER INDEPENDENT OR SHARED	SHARED VALUES
position	size	vertex alpha
	orientation matrix	blend mode (1)
	normal vector	blend mode (2)
	2D rotation	an RwTexture
	color	an RwMaterial
	vertex colors	the UseCenter values
	vertex 2 coordinates	
	vertex 4 coordinates	

One last distinction between these flags is that the names of shared values contain the letters "CNS" for constant or "shared". For each "CNS" variable there is an equivalent non-CNS value indicating that an independent value is given to each particle. The two settings are incompatible. For instance, the PTank can have one color for all particles or individual colors for each particle, but not both. It may have one size for all, or individual sizes for each, but not both.

Because of this flexibility it is possible for the developer to make a mistake and address an individual value, like the color of the tenth particle, when there is only a single color, stored as a shared value. The RpPTank warns about these errors in one of two ways.

If the developer addresses an individual particle's value, the address of the value must be found from `RpPTankAtomicLock()` and `RpPTankAtomicLock()` returns NULL when it is asked for the address of a non-existent value.

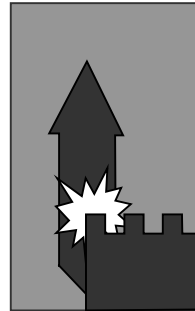
If the developer addresses an shared value, the data must be returned from a Get function. Each of the functions that get shared values return NULL if their values do not exist.

So it is worth checking the return values of all these functions.

The RpPTank is most efficient when it stores similar particles, but the developer may maintain very different particles by adding multiple RpPTanks, with a different organizations and different data.

18.2.2 The Particle Tank

A particle tank, or RpPTank, is an extended RpAtomic . Its Create function, RpPTankAtomicCreate() , appends space to the atomic to hold the data used to store particles. There is an equivalent RpPTankAtomicDestroy() that destroys the atomic. Because it is an atomic, it has location and a bounding sphere within the RpWorld and this allows the sprites stored in the PTank to be processed for visibility and rendering in the same system as the rest of RenderWare Graphics. This means that particles can be masked or hidden completely like other objects.



A particle may be placed behind and in front of objects in the RpWorld

Because a PTank is an atomic, it is necessary to be able to test whether a particular atomic is also a PTank, and the function RpAtomicIsPTank() returns TRUE if it is a PTank.

The particle tank simply contains the data for a fixed maximum number of particles. The maximum number is returned by RpPTankAtomicGetMaximumParticlesCount() and is set by the RpPTankAtomicCreate() function. But sometimes the application may not use the maximum number of particles in the PTank to avoid processing particles that are not seen. In such cases, the functions RpPTankAtomicGetActiveParticlesCount() and RpPTankAtomicSetActiveParticlesCount() can be used to get and set the lower value.

Some effects, like explosions and lightening strikes appear only briefly, so they should be considered for rendering only when their visual effect is required. The developer can switch them off by setting their number of active particles to zero and switch them on again by restoring the number of active particles. This saves processing time, and the function RpPTankAtomicGetActiveParticlesCount() is provided for this purpose.

The PTank is platform specific, so RpPTankAtomicCreate() takes a third parameter for platform-specific flags. Refer to a platform-specific API Reference for details of this parameter.

The PTank has one of two formats depending on the setting of the two flags listed above: rpPTANKKDFLAGSTRUCTURE or rpPTANKKDFLAGARRAY. It is either a structure (of arrays) or an array (of structures).

The diagram below represents four particles. They consist only of a vector and a color each. They are stored as an array of structures, "Structure Organization" and as a structure of arrays "Array Organization":



the integer, `stride` , is set to zero if the PTank is formatted in arrays. If it is formatted as an array of structures, the "stride" holds the size of the structure.

18.2.5 Locking and Unlocking

RenderWare Graphics usually requires the developer to

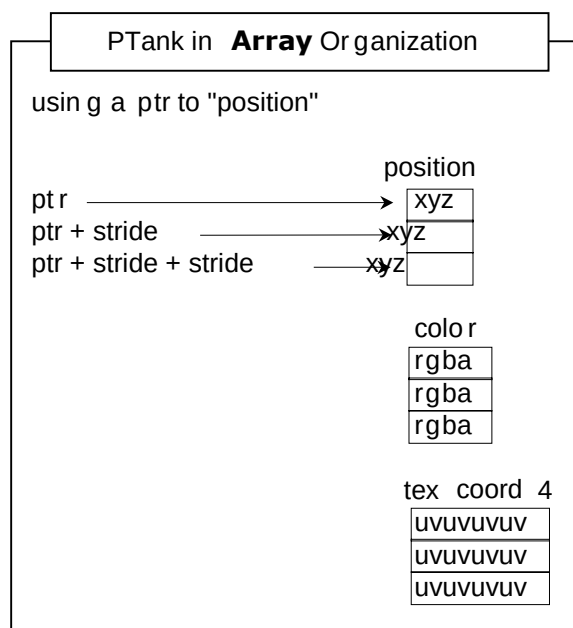
1. lock the data that is used for rendering before altering it
2. use the Get and Set functions to read or alter it
3. unlock it.

If you have locked data with the `rpPTANKLOCKWRITE` flag, the PTank has to re-instance the data for the current platform before rendering. So the developer should not use the Lock command needlessly. If it is accessed for reading only, it can be accessed with `rpPTANKLOCKREAD` to avoid this overhead.

The PTank plugin follows the same three steps, but the structure of the PTank makes it more efficient for the lock function to return a pointer to the locked data. So the second item in the sequence above is to address the data directly.

For this reason the PTank has many more values to be accessed than it has Set and Get functions. Under the heading *The Particle* above, there is a series of pre-processor constants that reserve space for various data items when a PTank is created. Each of the "`rpPTank LFLAG..`" constants listed there can be passed to the `RpPTankAtomicLock()` function to retrieve the address of its data so that the data must be read or written directly, without using Set and Get functions.

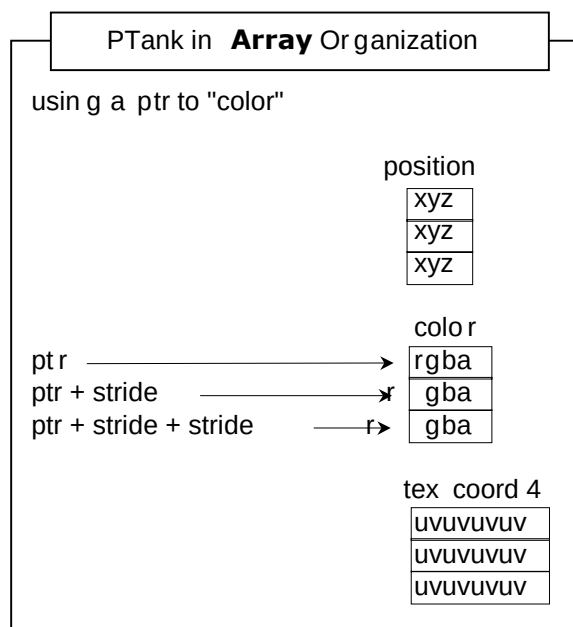
Whether the PTank uses Array Organization or Structure Organization, the developer can write code to read and write regardless of the organization.



Accessing PTank values in Array Organization

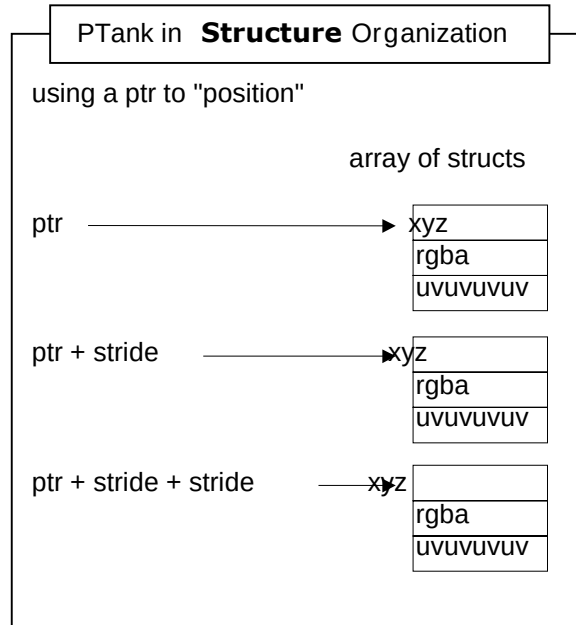
In the diagram above, the `RpPTankAtomicLock()` function has supplied a pointer ("ptr") to the first item of the position array. In "array organization" this is a pointer to an array of positions. The value "stride" is the size of the "xyz" vector, with any platform-specific padding.

In the diagram below, the same approach is used to read colors rather than positions. Only the initial value of the pointer is different.



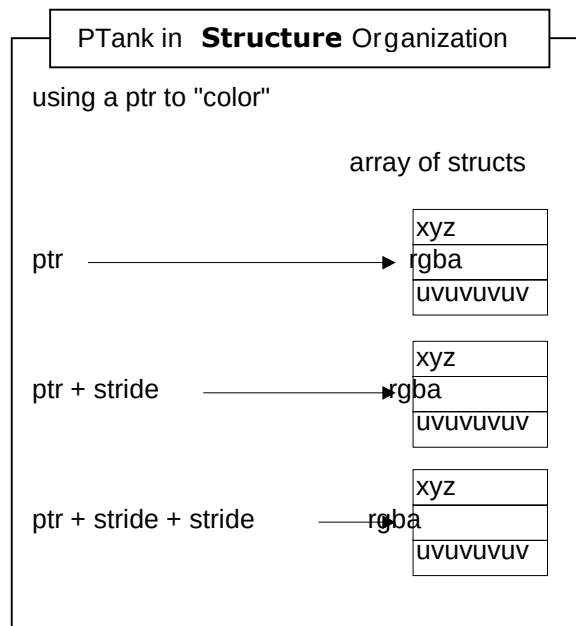
Accessing PTank values in Array Organization

The diagram below illustrates "Structure Organization", and the pointer refers to the first instance of the position in an array of structures. The value "stride" is the size of the structure, including any platform-dependent padding. So the value of the pointer, plus the value of stride is the address of the second position.



Accessing PTank values in Structure Organization

The same method works for colors, as illustrated, since the pointer points to the first color item, and pointer plus stride points to the second color item. And the same method can be applied to the four UV coordinates.



Accessing PTank values in Structure Organization

This means of addressing the data allows the developer to use the same code to calculate the pointer whether the PTank is an array of structures or a structure of arrays.

```
loop
    read the data at pointer (or write it)
    pointer += stride
end loop
```

This saves the developer duplicating code. But it allows the organization of the RpPTank to be determined automatically when the RpPTank is created, without the developer having to be aware which format was applied. If the RpPTankAtomicCreate() function is called without either the rpPTANKDFLAGSTRUCTURE or the rpPTANKDFLAGARRAY format flags being set, the format is chosen automatically. In this case, the developer need not know which organization is used, but the code will still address it correctly.

The advanced user may need to know whether the data is "structure organized" or "array organized". The answer can be found through the function RpPTankAtomicGetDataFormat() which returns the values in the RpPTankFormatDescriptor . The RpPTankFormatDescriptor contains the dataFlags and the dataFlags include the flags rpPTANKDFLAGSTRUCTURE and rpPTANKDFLAGARRAY. If the developer did not set these values in the RpPTankAtomicCreate() function, the RpPTank will have set them automatically, so the organization flags returned by RpPTankAtomicGetDataFormat() will accurately return the organization.

18.3 How to Use Particles Step by Step

These are the steps the developer needs to introduce particles into a developing application.

18.3.1 Initialization

Append the header file `ptank.h` to the list of included files.

Insert `RpPTankPluginAttach()` after `RwEngineInit()` and after attaching the World plugin, but before `RwEngineOpen()`.

Use the function `RpPTankAtomicCreate()` to create a PTank.

Set the number of particles that will be active with `RpPTankAtomicSetActiveParticlesCount()`.

18.3.2 Defining Particles

The function `RpPTankAtomicLock()` supplies a pointer to the data to write (or read) through the second parameter, called "dst" in the API Reference of the Lock command. The data pointer and the "stride" are supplied as elements of the dst structure.

```
RpPTankAtomicLock( pTank, &dst, rpPTANKDFLAGPOSITION,  
                  rpPTANKLOCKWRITE );
```

Loop through the PTank particles using the data pointer and adding "stride" to it at each iteration, as described under the previous heading. In this case, the data is a 3D vector and is cast to an `RwV3d *`, and it is transformed according to the state of other parameters.

```
if( dst.data )  
{  
    RwV3dTransformPoints( (RwV3d*) dst.data,  
                          &PositionsList[i], 1, Im3DmeshMatrix);
```

Write the required data to the address at `dst.data`.

```
    dst.data += dst.stride;  
}
```

The next step is to set the bounding sphere of the atomic. The developer may be used to using the function `RpMorphTargetCalcBoundingSphere()` but this is not suitable for particles. It is a safe starting point to set the sphere to be big enough to include everything. This will be inefficient and will have to be adjusted later. Also, particles usually present special effects and they are usually transient, so you will need to tune the bounding spheres for each effect. As there is only one bounding sphere for each PTank, you should consider grouping the particles by their position, in different PTanks. The bounding sphere is set with the function `RpMorphTargetSetBoundingSphere()` .

Finally unlock the PTank so that it can be used in the rendering process.

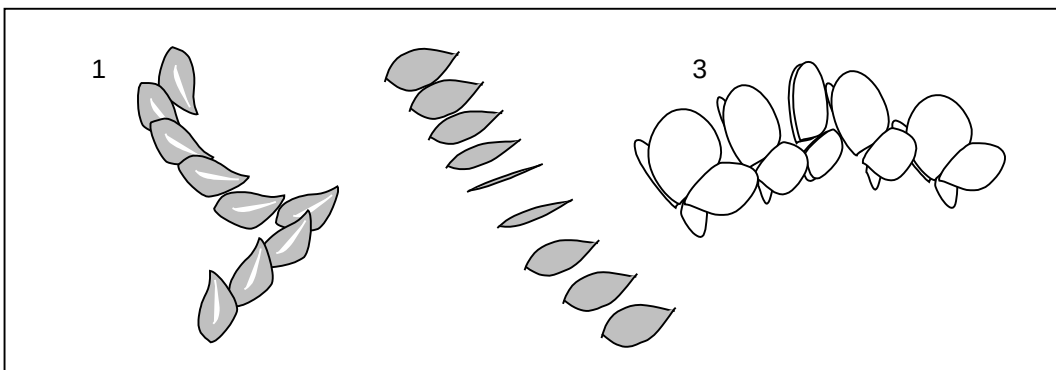
```
RpPTankAtomicUnlock( pTank );
```

As a minimum the user will have to define a position, a color and a size. This will define a visible rectangle in space.

There are other functions to define the particles displayed:

When a particle has to rotate (in 2D) the developer will often want to specify the point around which it will turn. This point will also be taken as its origin when it is positioned. `RpPTankAtomicConstantSetCenter()` sets this value but it will apply to all particles in the PTank.

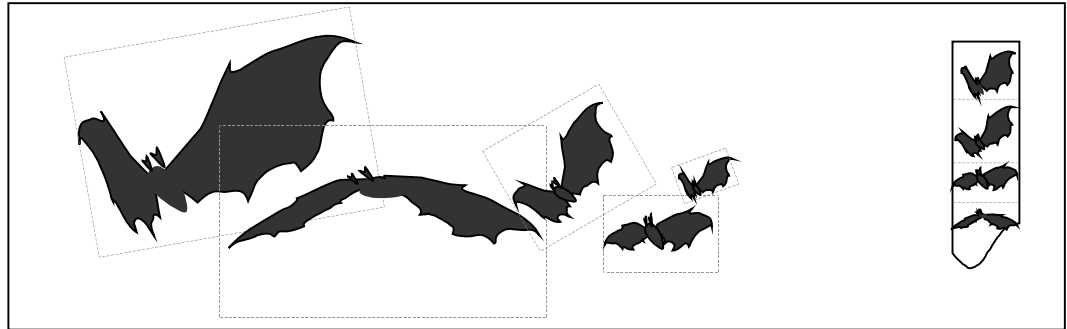
The function `RpPTankAtomicConstantSetMatrix()` sets a single matrix value that defines the orientation of all its particles to the screen. If there is a matrix for an individual particle it can be accessed with the `RpPTankAtomicLock()` function using `rpPTANKDFLAGMATRIX`



1. Rotation of a particle to represent a falling leaf
2. Matrix changes to give 3D rotation
3. Matrix changed on two particles to animate a butterfly.

The function `RpPTankAtomicSetConstantRotate()` sets the angle of rotation (in radians) for all the particles in the PTank. If there is a rotation value for each particle in the PTank it can be accessed by the Lock function passing the constant `rpPTANKDFLAGROTATE`

The function `RpPTankAtomicSetTexture()` sets the `RwTexture`, which may represent any 2D image for a given particle within the PTank. This can be used to display images from artists' tools on the particles. The alpha value can be used to make them partially transparent, or to alter the visible outline of the particle. (This method is implied in the two leaves and the butterfly in the illustration above.)



Separate areas of an `RwTexture` can be applied successively to a particle to animate its image

The function `RpPTankAtomicSetConstantVtx2TexCoords()` sets the UV values to be given to the top left and bottom right coordinates of the particle. An `RwTexture` will be applied to its surface with the coordinate values given as parameters mapped to points 0,0 and 1,1 on the particle. This can be used to map different section of the texture successively to the particle, rather like displaying successive frames of a film, for animation.

The function `RpPTankAtomicSetConstantVtx4TexCoords()` sets the UV values to be given to the four corner coordinates of the particle. An `RwTexture` will be applied to its surface with the coordinate values given as parameters mapped to points 0:0, 0:1, 1:0 and 1:1 on the particle. This too can be used for animation but is better used to stretch the texture out of its default, 1:1 aspect ratio.

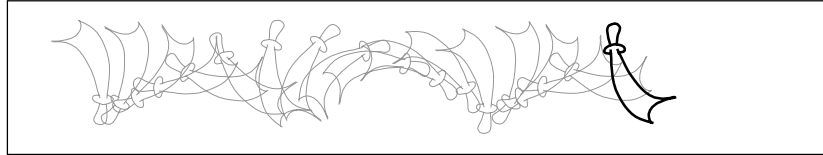
The function `RpPTankAtomicSetConstantVtxColor()` passes an array of four colors for the four corners of the particle. The color between is blended in proportion to its distance from each corner.

The function `RpTankAtomicSetVertexAlpha()` sets the opacity of all the particles in a PTank, and is useful for setting the transparency of smoke, clouds and ghosts.

18.3.3 Animation

For some effects the user may want simple animation.

An particle can be moved by updating its position. If its size is updated it will appear to get bigger or smaller, and therefore to move to or away from the camera.

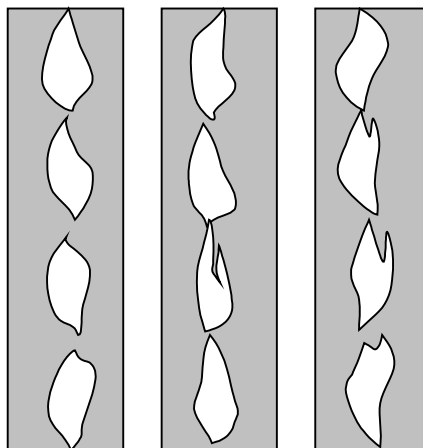


Position and rotation can be updated to give animation

If an image needs to be rotated like a scimitar thrown across the field of view, this can be achieved by updating the position and rotation value of the particle alone.

A particle can be rotated out of screen-alignment like turning a page of a book. A few useful effects may be achieved this way.

The vertex alpha value (or opacity value) of a particle can be altered to make it appear slowly like a ghost, or to make it disappear, like rising smoke, or the cloud that represents an explosion. The vertex alpha makes the whole particle increasingly more transparent or opaque, between its rgba values and total transparency. It is addressed by the function `RpPTankAtomicSetVtxAlpha()` , and affects all particles in the PTank.



Flames can be animated as a series of images using `RwTextures`

An effect rather like movie film animation can be achieved by producing an `RwTexture` that resembles a film, consisting of successive images, like the frames of a movie. A particle is rendered with successive areas of the texture mapped to its UV coordinates. This is similar to projecting successive frames of a movie onto it, with the extra feature that the image can have transparency and adopt shapes other than a rectangle. This approach is useful for effects like flames, sparkle and specters.

18.4 Examples

Three Examples of RpPTank are provided called "ptank", "ptank2" and "ptank3".

The first "ptank" displays particles arranged as if on the surface of a rotating doughnut. All the particles in the Example are identical. The user may change several parameters interactively from the menu and some from the code.

"Ptank2" is based on the previous example. It adds the ability to rotate the particles and allows the user to alter their control parameters.

"Ptank3" is also based on the previous example and adds `RwTexture` to the particles and the user can adjust their parameters interactively.

18.5 Troubleshooting

Use the debug version of the code. It will give an `rwDEBUGASSERT` when it finds potential conflicts, like incompatible flag settings.

If values are written to particles but have no effect, or garbage values are returned, remember that PTank values exist in alternative forms. They may exist either as a single value shared by all the particles in the PTank, or as independent values for each particle in the PTank. Some values don't exist in all PTanks. If the developer addresses non-existent values nothing will happen and nothing will crash. But the constant values are address via functions that return NULL if invalid values are addressed, and all independent values are accessed via the `Lock` function. The `Lock` function will return NULL if it is asked for the address of a non-existent value. So make sure that these return values are checked.

Some functions take pointers to an array of colors or vertices. If the array has too few elements the results will be unpredictable.

Particles will not be rendered if they are outside the bounding sphere on their PTank. Bounding spheres need to be set by the developer, not by `RpMorphTargetCalcBoundingSphere()` when using particles. To see if the bounding sphere is clipping a particle, define an enormous bounding sphere for the particle's PTank.

18.6 Summary

Particles are simple, flat images, like sprites. They are much simpler to render than images that need the full 3D rendering processing. But they do exist in space in RpWorld and will be masked by objects in front of them.

Particles can be scaled, rotated, positioned and moved. They can have a texture applied to them. Their whole image can have a transparency level, so they can appear or disappear gradually, and their texture image can have transparency levels so they can appear as different shapes.

Particles are good for fleeting effects like explosions and for fast-moving or small objects that don't justify detailed 3D rendering.

The PTank or Particle Tank, is the object that stores particles. PTanks can have varied formats. Some values are held in common between all the particles in a PTank other values are stored independently for each particle. Multiple PTanks can be used at the same time to support widely different particles appearing concurrently.

There are two major variations on the internal organization of data in a PTank. The developer may chose the more efficient form or may allow PTank to decide. The code may be written to address the two different formats efficiently without the developer necessarily knowing which format is adopted.

By default, particles face the camera or cameras. Images that don't suit this form of image are probably not appropriate for particles. But they can be oriented relative to the screen plane. Lettering, titles and other 2D graphics are well supported in the Rt2D toolkit.

Snowstorms, galaxies, realistic clouds, flowing water and the more spectacular effects associated with particles require dedicated animation software.

The locking function for particles differs from other elements of RenderWare Graphics in returning a pointer to internal data. The user updates the pointer and addresses the data directly. So, many properties of a particle do not have their own Get and Set functions.

Three pieces of Example code are provided for the user to compile and experiment with.

Particles can be used to produce animation by a few simple techniques. Most versatile is the use of successive images applied as textures rather like the successive frames of a movie.

Chapter 19

B-splines and Bézier Patches

19.1 Introduction

This chapter falls into three sections.

The first describes the way RenderWare Graphics adapts B-splines to deal with curved lines and paths, in the `RpSpline` plugin.

The second section deals with the way RenderWare Graphics encodes curved surfaces, through Bézier patches. It explains some of the methods and the API calls which render patches in the `RpPatchMesh` plugin.

The third section covers the `RtBezPat` Toolkit. Using the Toolkit, the developer who wants to get more out of the code can call the most useful functions hidden below the surface of the `RpPatchmesh`. It includes functions that find matrices of vectors for positions and tangents and other functions that speed up calculations. These calls require some knowledge of the mathematics of curved 3d surfaces, and they are a valuable resource in their own right.

Other Documents

See the API reference for details of the code for `RpSpline`, `RpPatch` and `RtBezPat`.

The Appendix, Recommended Reading, lists sources that give background in this subject.

This chapter assumes you have some familiarity with the concepts of materials, skinning, atomics, clumps and streaming from their descriptions in this User Guide. They can be found in the table of contents. In particular, it is assumed that the reader knows the Chapter on Dynamic Models. The patch mesh code is designed to integrate with the other elements of RenderWare Graphics. So the same methods and API calls that were used with the flat polygons, in Dynamic Models, are re-used in the patch mesh section of this chapter to control the rendering, lighting and texturing of surfaces which are curved. Functions and data are named to parallel those used earlier, and the code example `patch`, used in this chapter, is based on the earlier example, `geometry`. (For completeness it should be noted that one area of functionality of `RpGeometry`, morphing, is not implemented for patches.)

Several technical and mathematical topics touched on in this chapter are covered on specialist web sites and can be found by a web search.

19.2 B-splines

19.2.1 Introduction

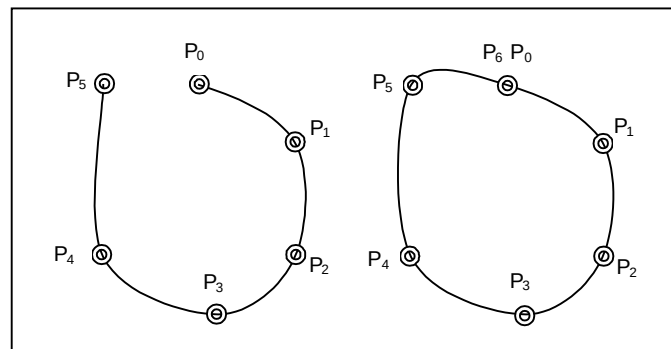
RenderWare Graphics uses uniform B-splines to define paths for cameras, lights and other objects. The `RpSpline` plugin implements B-splines and it is described here.

19.2.2 What Are B-splines?

The third degree B-splines that RenderWare Graphics uses are defined by a sequence of control points in 3d space. The sequence can have four or more control points. The position of any point on the B-spline that occurs between two control points in the sequence is determined by the two control points before and after it in the sequence; four points in all. Only the first and last points in an open B-spline sequence are on the curve. The curve bows towards each of the other control points in turn, but does not normally go through them. Expressed in intuitive terms, the curve is attracted to each of its control points in turn.

Control Point Numbering

In `RpSpline`, a B-spline's control points are numbered beginning from P_0 . By convention P_{n-1} refers to the last point. In a closed spline, where the first point is also the last point, both P_0 and P_n refer to the same control point.



Numbering of `RpSpline` control points

Internally `RpSpline` may add extra control points at the start and end of the spline but this is transparent to the developer and does not affect the control point numbering in the API functions.

If a B-spline is defined by points numbered P_0 , P_1 , P_2 to P_n , then the points on the curve segment P_i to P_{i+1} are calculated from points P_{i-1} , P_i , P_{i+1} and P_{i+2} . The formula for these segments applies along the whole curve. It even applies on the first segment P_0 to P_1 , and the final segment, P_{n-2} to P_{n-1} , because `RpSpline` intercepts the numbering of these control points as explained under Control Points at Open Ends, below.

19.2.3 Some Features of B-splines

Smoothness

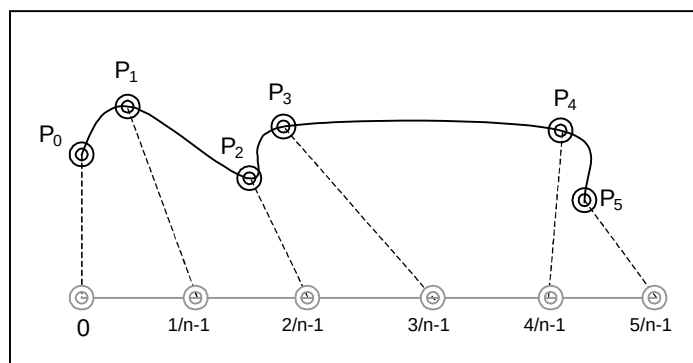
Perhaps the most common types of spline in computer graphics are B-splines, Bézier curves and the closely related Hermite curves. B-splines are rather smoother or rounder than the more angular Hermite curves, and the Bézier curves that are used in RenderWare Graphics' patches.

On-curve and Off-curve Control Points

Control points P_0 to P_{n-1} are an integral part of a B-spline and its mathematics, but on-curve points are more intuitive and more useful for many purposes. So RenderWare Graphics incorporates an algorithm to convert efficiently between off-curve points and their corresponding on-curve points and vice versa. So, as far as the developer is concerned, all the control points that RpSpline uses are on-curve points. In this section on RpSpline only, this document will refer to control points as if they were all points on the B-spline.

Real-world Space and Parameter Space

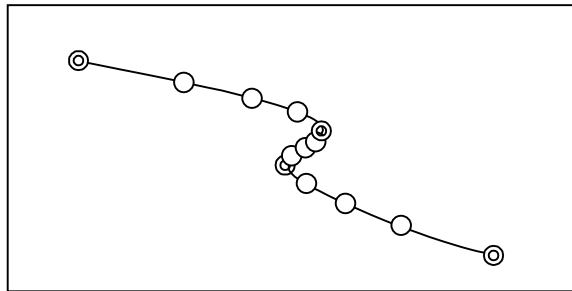
A B-spline, like other types of spline, can be seen as a sequence of points in real-world space, that define a curved line. For calculation purposes the B-spline can also be considered as a 1d progression where the control points are spaced uniformly or irregularly along its length. This second representation is referred to as "parameter space". In parameter space a uniform B-spline's control points are spaced uniformly (or evenly) in parameter space, at intervals of $1/(n-1)$ for a curve with open ends, and $1/n$ for a curve that forms a closed loop back to its starting point. Each point of the B-spline represented in parameter space corresponds exactly to one in the real-world B-spline.



Real-world space corresponds to parameter space

Velocity

The control points that define a curve can be spaced far apart or close together, in real-world space. If an application plots the progress of an object at intervals derived from the spacing of the control points in real-world space, the object's speed will be slower on the tighter bends, where the control points are closer. This can be useful because it can correspond to the behavior of racing cars on a circuit and to the number of straight segments or facets needed to represent curves where the curvature is tighter.



Control points, and points derived from them, are closer where the spline curves more steeply.

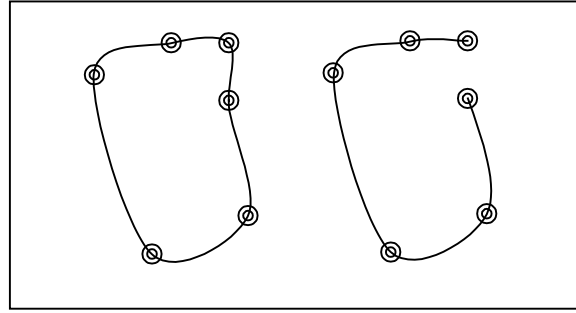
This is not the sense in which "velocity" is used with `RpSpline`; the developer decides how often to sample the curve and re-draw the object moving along it. But `RpSpline` introduces "velocity" in two enumerated values that affect the behavior at the ends of B-splines.

If the curve represents a path for a solid object it may also be useful to specify that the object accelerates at the start and decelerates toward the end. This feature is described in *RenderWare Graphics* as velocity. The values `rpSPLINEPATHSMOOTH` and `rpSPLINEPATHNICEENDS` can be passed to the functions `RpSplineFindMatrix()` and `RpSplineFindPosition()`. The first returns values for the developer who wants progress to be defined in parameter-space intervals that are even, and the second returns parameter-space intervals that are spaced to accelerate and decelerate towards the ends of the spline. The developer may choose to apply these to the spline's real-world.

`RpSpline` will support "constant velocity" only if a B-spline's control points are spaced at equal intervals.

Open and Closed B-splines

A curve can be "closed" in a continuous loop, or it may have two "open" ends.



A "closed" and an "open" Rpspline

Any curve can come back to its first point, but to say that a curve is "closed" usually means that the join is just as smooth as the curve at each of the other control points. This is seen in the diagram above, where the direction of the line at the start and end points changes according to whether the curve is open or closed.

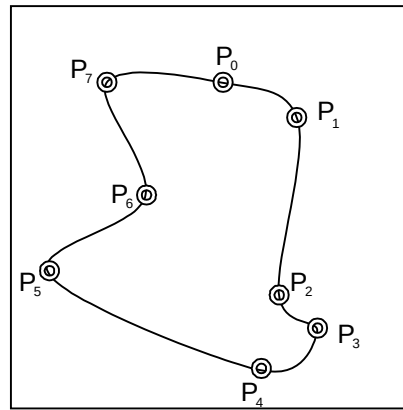
Control Points at Open Ends

Rpspline has a problem with open curves. The first and last segment do not form a sequence of four control points of which the segment runs between the middle two. Rpspline simplifies things by introducing two extra points. It does this behind the scenes adding an extra first and last point to each open curve, at the start and the end of the curve. These two points have exactly the same position as the first and last points, but they make it possible to apply the same "base function" to the first and last segments and provide a suitably smooth start and end of the curve.

These points are introduced and processed transparently and do not affect the control point numbering.

Closed B-spline Closed Curve Joins

In order to join a curve P_0 to P_7 in the diagram below, Rpspline treats P_0 and P_1 as points P_8 and P_9 as if they followed P_7 at the end of the spline. In this way it can apply the same function as it applies to all the curve segments in the spline. If the current curve segment begins at point i , then the function will take the same parameters, P_{i-1} , P_i , P_{i+1} and P_{i+2} , and this function will calculate the curve that joins the end points so that it is just as smooth as the curve between the other control points.



An eight-point closed RpSpline

At the beginning of the curve it does much the same thing, inventing a P_1 and P_2 which coincide with the last point (P_7) and the point before that (P_6).

An RpSpline has n control points, and they are numbered from P_0 to P_{n-1} . RpSpline calculates the last segment of a closed B-spline, the segment from P_{n-1} to P_0 , by using points P_{n-2} , P_{n-1} , P_0 and P_1 . Similarly the first segment P_0 to P_1 is calculated using P_{n-1} , P_0 , P_1 and P_2 . Thus a closed curve always joins smoothly.

19.2.4 Why Use B-splines?

There are other curve formats and they have different strengths and weaknesses, but B-splines are now well understood and their mathematical problems are largely solved. They allow us to calculate tangents. Long splines of many points are continuously smooth by their nature, without the need to smooth the joins of many shorter curve segments. And sometimes it's convenient to convert them mathematically to or from Bézier and Hermite curves.

B-splines have "local control". That means that if a control point is moved in order to move part of the spline, it affects the path of the spline no further than two control points behind or ahead in the sequence. So if point P_i is moved, the B-spline will be affected as far as P_{i-1} and P_{i-2} and in the other direction, as far as P_{i+1} and P_{i+2} . In other types of spline, the curve can be affected one or more control points further in both directions.

Other curve formats have the advantage of being easy to control by specifying their start and end points, and they move toward their off-curve control points in a way that is fairly intuitive. RpSpline has a greater advantage in that the control points all lie on the curve, so the spline goes exactly where the control points specify.

The convenience of making the curve join back on itself in a continuous loop without extra code or extra calculations is also an advantage.

Basic Mathematics

Earlier pages in this chapter referred to some processes that RpSpline performs internally. RpSpline uses matrix inversion to convert between its on-curve control points and the off-curve control points used in the conventional mathematics of uniform B-splines. We have seen how RpSpline alters the control point sequence so that the ends of loops can use the same function, so the first and last curve segments join as smoothly as the other curve segments.

In effect then, RpSpline can apply the same formula to all curve segments. This can be implemented in a single "basis function" or "smoothing function". The two control points in the control point sequence that enclose the curve segment are described as P_i and P_{i+1} . The basis function uses the four control points P_{i-1} , P_i , P_{i+1} and P_{i+2} to calculate any position on the spline between P_i and P_{i+1} . The value "u" increases linearly from zero to one representing positions on the spline between P_i and P_{i+1} . The formula that calculates positions on the spline is

$$P_{i+2} u^3 + P_{i+1}(3u^2 - 3u^3 + 3u + 1) + P_i(3u^3 - 6u^2 + 4) + P_{i-1}(3u^2 - u^3 - 3u + 1)$$

6

It applies varying proportions of the positions of each of the four control points to the point calculated. The amounts vary according to how far the point is from P_i to P_{i+1} .

19.2.5 How RenderWare Graphics Processes Two-dimensional B-spline Curves.

The routines to handle B-splines are contained in the RpSpline plugin. The RpSpline plugin must be attached using RpSplinePluginAttach() before an application can use the functionality described in this chapter. The attach function must be called after RwEngineInit() and before RwEngineOpen(). The header file rpspline.h must be included in files that use these functions and structures.

Struct RpSpline

RenderWare Graphics maintains a datatype, RpSpline, to handle each B-spline. It stores:

file and path name (a null-terminated array of RwChar)

control points (a pointer to an array of RwV3d)

number of control points (an RwUInt32)

and the spline type, including information like whether the spline is open or closed (an RwUInt32)

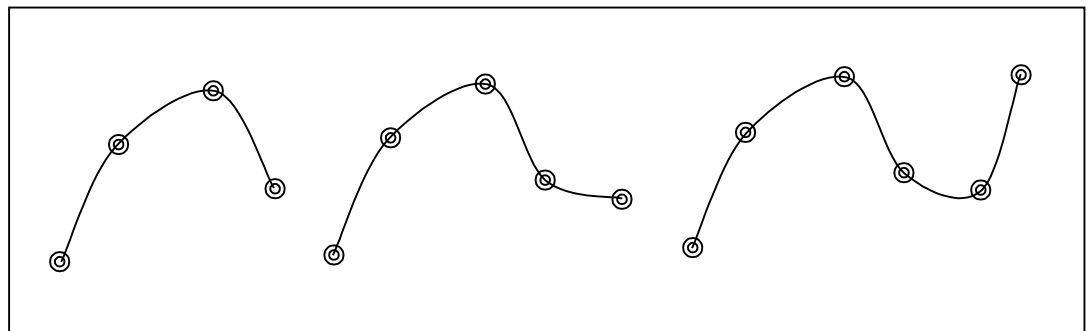
and other data.

API functions are provided to address the structure and maintain its integrity. The developer should not address its members directly. Its members are listed in the API reference.

Creating a Spline

The function `RpSplineCreate()` takes three arguments. If it is successful, it returns a pointer to a new `RpSpline` structure.

The first argument, an `RwUInt32` gives the number of control points. This must be four or more.



B-splines defined by 4, 5 and 6 points

The second argument indicates whether the spline is to represent an open curve, or whether it is a closed loop. `rpSPLINETYPEOPENLOOPBSPLINE` and `rpSPLINETYPECLOSEDLOOPBSPLINE` specify the type of spline.

The third argument is a pointer to an array of three dimensional vectors, a `* RwV3d`, which constitutes the sequence of control points.

Cloning

`RpSplineClone()` provides an easy way to create or copy splines. The function takes only one argument, a pointer to the spline to clone. It returns a pointer to the new spline, which has a pointer to a new copy of the control points so that the new spline is entirely independent of the first.

A spline which has been created or cloned can have its control points adjusted by `RpSplineSetControlPoints()`. This function takes a pointer to the spline to adjust, an integer (≥ 0) to specify which control point is to be adjusted and a pointer to a three dimensional vector that specifies its new position. It returns a pointer to the new position, or `NULL` if it is unsuccessful.

Finding Frames, Positions and Control Points

A B-spline may be used to define a path. If it is the path of a racing car, then the developer will need the tangent of any point on the path in order to point the racing car in the right direction at that point. The function `RpSplineFindMatrix()` returns the position of the point and its direction.

`RpSplineFindMatrix()` takes five arguments, of which four pass data to the function and one returns it. The first argument is a pointer to the spline. The second is a bit-significant `RwUInt32` that describes the type of path as of constant velocity, with `rpSPLINEPATHSMOOTH` or with acceleration at both ends, with `rpSPLINEPATHNICEENDS`. The third is an `RwReal` that specifies how far along the path (from P_0 to P_{n-1}) the point is as an `RwReal` between zero and $n-1$. And the fourth is a pointer to the frame's look-up vector. The last argument is a pointer to the Frenet matrix that can be used to orient the frame of an object at the given point on the spline. The function returns an `RwReal` that is the Gaussian description of the tightness of the spline's curvature at the point specified.

`RpSplineFindPosition()` finds the position in space of a point somewhere along a given B-spline. It also returns its direction at that point.

The function takes five arguments. The first three pass data to the function the other two are used to return data. The first argument is the address of the spline, the second is a bit-significant `RwUInt32` that defines the type, specifically the velocity of the path, and the third is the 'u' value from 0 to 1 which determines how far along the path (from P_0 to P_{n-1}) the specified point occurs (so the i^{th} control point is at i/n).

The path argument is an integer equal to `rpSPLINEPATHSMOOTH` or `rpSPLINEPATHNICEENDS`. The first will take the value to mean that 1/3 and 1/6 correspond to the control points, even if the first three control points are close together and far away from the fourth. The second defines the start and ends with increasing resolution which accelerates and decelerates between the first two and the last two points of the spline.

The fourth and fifth arguments are `RwV3d` vectors to receive the position of the object, expressed in world space, and the tangent of its direction. These allow the frame of any object to be found on the curve at the angle queried. The return value is the position along the curve, or null if the function fails.

The function `RpSplineGetNumControlPoints()` takes a pointer to the `RpSpline` and returns an `RwUInt32` that specifies the number of control points.

The function `RpSplineGetControlPoint()` returns the position of a specified control point on the specified spline. It takes a pointer to the spline, a zero-based integer to define the control point and a pointer to an `RwV3d` that will receive the position of the control point. The return value is the position vector.

The complement of this function is `RpSplineSetControlPoint()`, which takes a pointer to the spline, a zero-based integer to specify the control point and a pointer to an `RwV3dvector` to specify its position. It returns `NULL` on error, and the pointer to the spline if it is successful.

Destruction

The function `RpSplineDestroy()` destroys the `RpSpline` structure as it is created by `RpSplineCreate()`.

Serialization

Four functions support reading and writing `RpSpline` data. `RpSplineRead()` takes a pointer to the file's path and file name and returns a pointer to the spline data that it has read in, or `NULL` if it fails. `RpSplineWrite()` takes a pointer to the `RpSpline` that it is to write, and a pointer to the file name.

Two other functions `RpSplineStreamRead()` and `RpSplineStreamWrite()` read and write to a RenderWare Graphics binary stream. They both assume that the stream is already opened.

The user may find out how much memory an `RpSpline` occupies, to check that there is enough disk space to hold the data.

19.2.6 Spline Summary

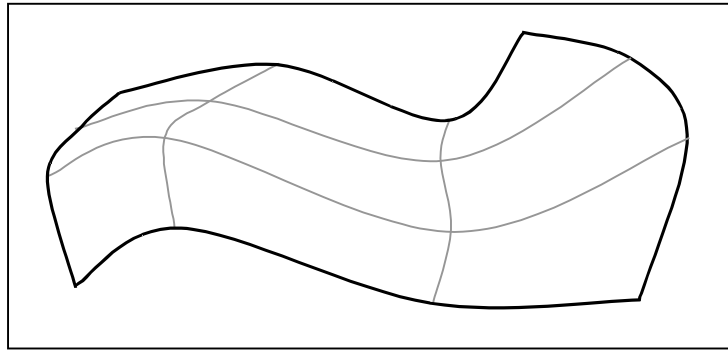
`RpSpline` is based on uniform B-splines. In RenderWare Graphics, they have on-curve points that appear to work as control points. They can be closed loops, in which case they are made to join smoothly automatically, and they can return their direction and position at any point. There is a full range of functions to stream them to or from files.

The rest of this chapter is concerned with patches, which are 3d shapes not curves. In RenderWare Graphics they are based on the mathematics behind Bézier curves rather than B-splines.

19.3 3D Bézier Patches

19.3.1 Introduction

One method of rendering a solid object for the computer screen is to divide the object's surfaces into polygons, usually triangles. Then the color of each visible triangle is calculated and each triangle is drawn onto the screen. This approach needs lots of triangles to represent curved surfaces and it still leaves surfaces looking unacceptably faceted when shown in close detail. RenderWare Graphics solves this by using patches.



A patch defined by eight curved lines.

The `RpPatch` routines use Bézier patches, shapes that curve in three dimensions and are bounded by Bézier curves on each side. They correspond to the simpler `RpGeometry` used elsewhere in RenderWare Graphics, that uses flat polygons bounded by straight lines.

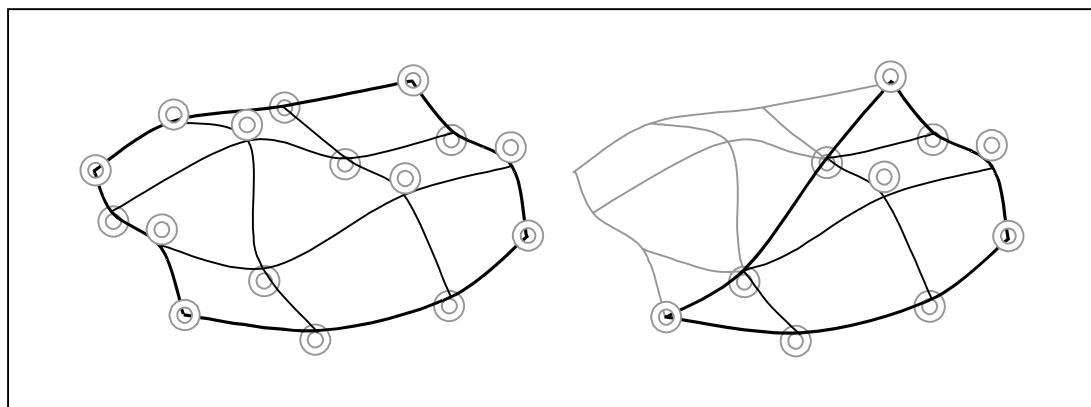
This middle section of the chapter describes how the `RpPatch` plugin implements several processes to take advantage of curved patches and integrates them with rendering processes; the developer needs to know only the broad framework of how they work.

19.3.2 What Are Patches?

In RenderWare Graphics, patches are quadrilateral or triangular shapes that curve in three dimensions. Each side of the shape is a Bézier curve. The patches are defined by control points like those which define Bézier curves and similar to the off-curve control points of B-splines. These control points define the Bézier curves that define the edges of the patch. But patches calculate the continuous surface from the control points whereas the simpler mathematics of Bézier curves need only calculate a single line.

Quad Patches and Tri Patches

A patch with four sides is called a "quad patch". Each side is a Bézier curve and therefore has four control points. This implies four other curves (shown in the diagram in gray) linking control points on the edges. This results in four curves running across the patch in one direction, and four more crossing them in the other direction. These curves cross at 16 points on the surface corresponding to the 16 control points of the patch. RenderWare Graphics calculates the continuous surface defined by these 16 control points.



A quad patch with 16 and a tri patch with 10 control points

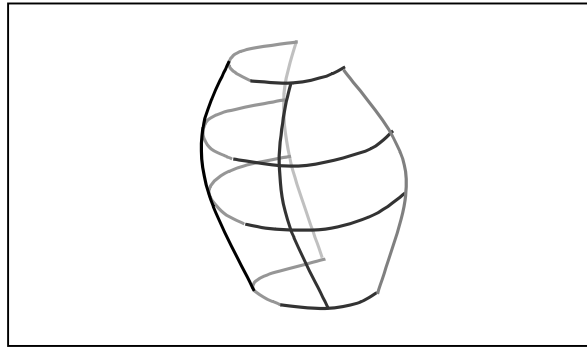
A patch with three sides is called a tri patch. Its three sides are Bézier curves, so each side is defined by four control points.

A control point at the patch's corner defines the corner's position. But the other points are not on the surface they define. So, in a quad patch, 12 control points are normally above or below the surface they define. In a tri patch seven control points are off the surface. To describe it more intuitively, it is as if the surface is attracted towards the other control points but fixed only to its corner points.

19.3.3 Why Use Patches?

It is clear that patches are more complicated than flat polygons. They have more control points and the control points are inter-related. They need more complicated mathematics and more calculations to derive useful information from them. The code to do this is proportionately harder to develop and to integrate with the rest of the rendering system. Different platforms support patches to very different extents, and RenderWare Graphics must accommodate the differences to support each of them efficiently. So, why use patches?

Each patch takes more data to define it, but a single patch can replace very large numbers of flat polygons, reducing storage space and processing time.



An example of a single patch that would take many triangles to approximate

In practice, these patches are rendered as small triangles, but it is the graphics processor that refines them, so it doesn't slow the main processor or use excessive amounts of memory.

Patches have another advantage; they can be divided into many small triangles for rendering, or, just as easily, into a few big triangles. So they can be rendered at varying levels of detail. Usually software improvements involve a trade-off. But introducing patches allows RenderWare Graphics to reduce render time and at the same time to reduce memory usage and still improve visual quality, all to produce the optimal visual effect.

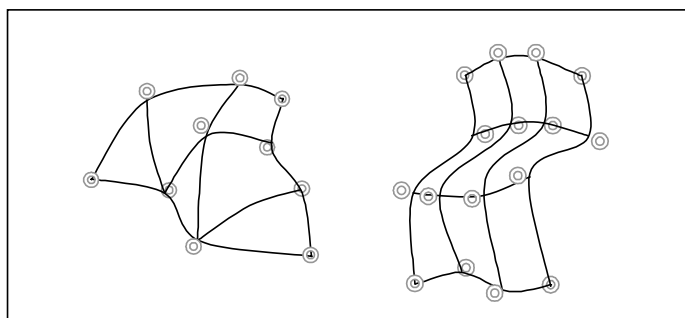
It is possible that future graphics processors will handle patches directly, either as curved surfaces, or by splitting them into facets. If they do then patches will give even greater efficiency to the rasterization process.

19.3.4 How RenderWare Graphics Handles Patches

RenderWare Graphics defines two types of patch:

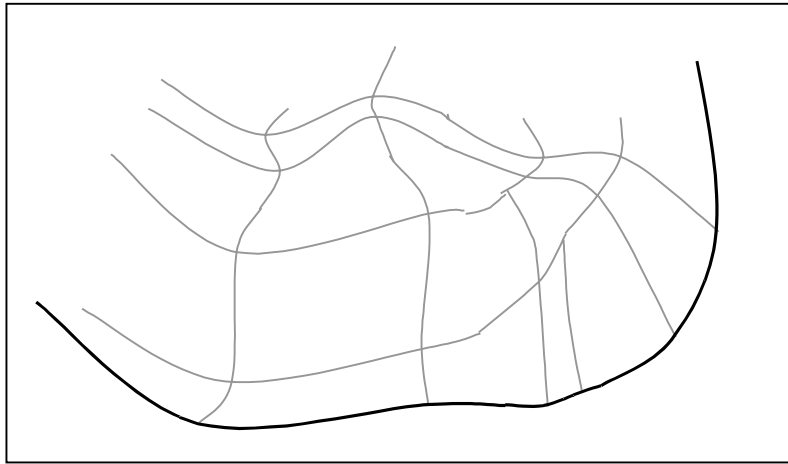
`RpQuadPatch` represents a quad patch as 16 indices to control points

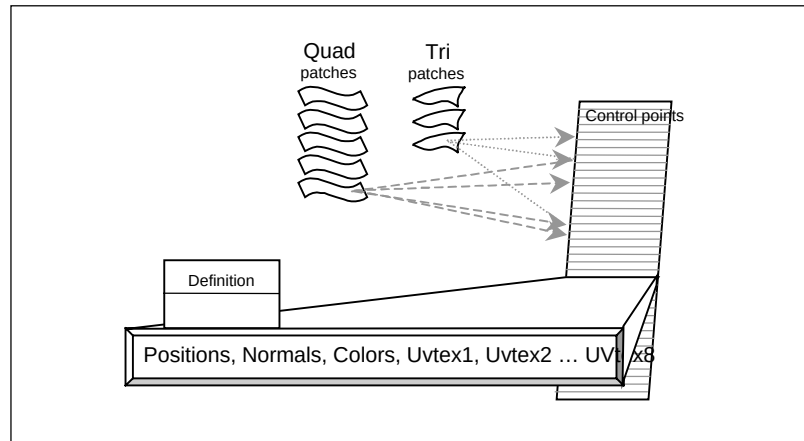
`RpTriPatch` represents a tri patch as 10 indices to control points



An tri patch with 10, and a quad patch with 16 control points.

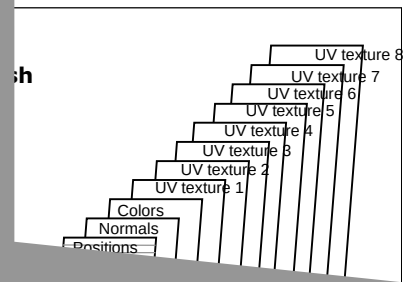
Both types of patch are stored in an `RpPatchMesh` to record the way the patches fit together. A single patch describes only a simple curved surface, but when patches are fitted together in a patch mesh they can define complicated surfaces.

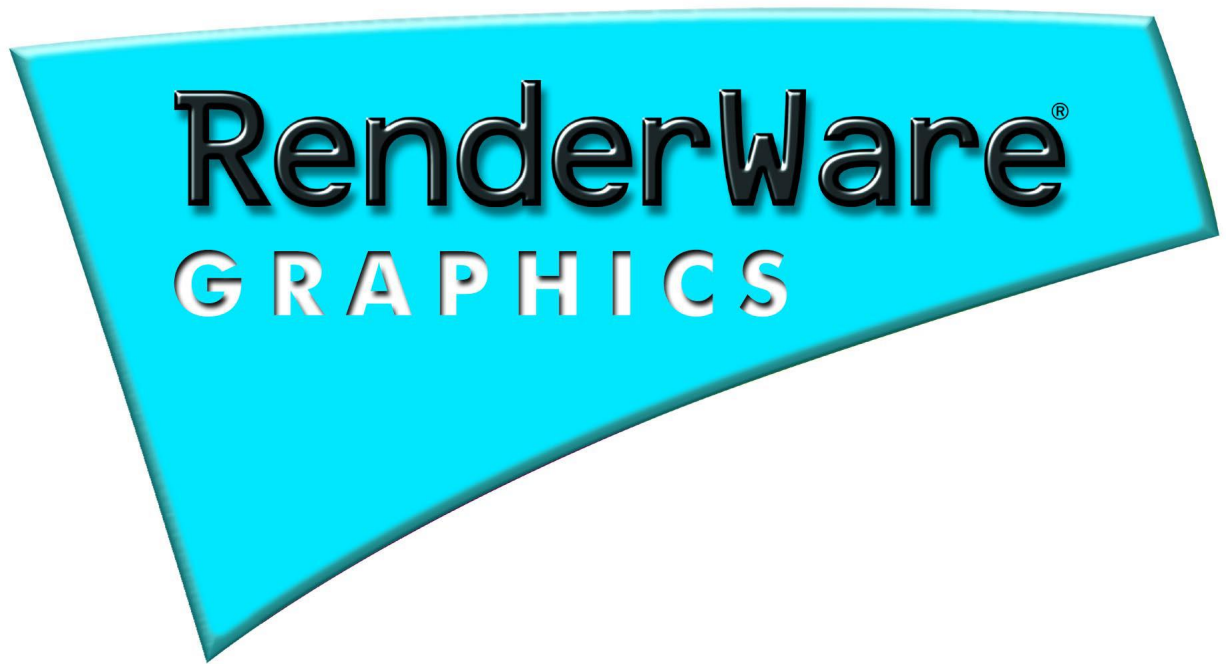


High-level view of `RpPatchMesh`

The control point in this sense is flexible. It has its own `RpPatchMeshDefinition` structure and that determines which data the control point holds. This may include the coordinates of its position, its pre-calculated normal vector, its pre-light color (its color before it is lit) and various sets of coordinates to stretch textures over it. But these can be present in any combination that the developer requires. (In principle none of them needs to be present.) This is the underlying concept and the most helpful way to see the patch mesh. In this sense a control point would probably have positions and normals to define the shape of the patch mesh. However, the same patch mesh could be defined with its control points containing only positions, if the normal vectors are not required.

The C language does not represent `RpPatchMesh` so economically, as is shown in the diagram below. The diagram illustrates only part of the `RpPatchMesh` and much of it is "opaque", or hidden from the developer. Where API calls are provided to access it, the developer should use them to maintain the consistency of data (and because the API functions will assert if they detect invalid data).





User Guide

Volume II

SDK Release 3.3